industrial-engineering: the Industrial Engineering and Supply Chain program, user guide

industrial-engineering-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Industrial Engineering and Supply Chain covers inventory policies (EOQ, reorder point, safety stock, newsvendor), statistical process control, reliability, queueing, scheduling, forecasting accuracy, line balancing, learning curves and facility location. This guide is about `industrial-engineering`, the model as a self-contained program for Windows and Linux: 94 capabilities in 11 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/industrial-engineering-ai999x/); the Python package (`pip install industrial-engineering-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.5.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Industrial Engineering and Supply Chain	Windows x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-industrial-engineering-1.5.0-windows-x86_64.zip
Desktop app: Industrial Engineering and Supply Chain	Linux x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-industrial-engineering-1.5.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-1.5.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-1.5.0-linux-x86_64.tar.gz

what	system	download
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-cli-1.5.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.5.0/ai999x-cli-1.5.0-linux-x86_64.tar.gz

Desktop app

Industrial Engineering and Supply Chain in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-industrial-engineering-1.5.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-industrial-engineering.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.5.0/ai999x-industrial-engineering-1.5.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-industrial-engineering-1.5.0-linux-x86_64.tar.gz
./ai999x-industrial-engineering-1.5.0-linux-x86_64/ai999x-industrial-engineering
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.5.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.5.0/ai999x-1.5.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.5.0-linux-x86_64.tar.gz
./ai999x-1.5.0-linux-x86_64/ai999x
```

Command line, Linux

The bundle holds one program per model; `industrial-engineering` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.5.0/ai999x-cli-1.5.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.5.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.5.0-linux-x86_64:$PATH"' >> ~/.bashrc && source ~/.bashrc
industrial-engineering --version
```

Command line, Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.5.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.5.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.5.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.5.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
industrial-engineering --version
```

In your browser

Open https://www.ai999x.com/models/industrial-engineering-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install industrial-engineering-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ industrial-engineering --version
industrial-engineering-ai999x 1.5.0 (Rust build, ax-engine 1.5.0)
```

2.2 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ industrial-engineering search "economic order quantity (eoq)"
ie_eoq                              Economic order quantity (EOQ)  (inventory / EOQ)
ie_epq                              Economic production quantity (EPQ)  (inventory / EOQ)
ie_eoq_quantity_discount            EOQ with all-units quantity discounts  (inventory / EOQ)
ie_inventory_total_cost             Annual inventory cost of an order quantity  (inventory / EOQ)
ie_eoq_backorders                   EOQ with planned backorders  (inventory / EOQ)
ie_newsvendor_normal                Newsvendor order quantity (normal demand)  (inventory /
newsvendor)
ie_newsvendor_poisson               Newsvendor order quantity (Poisson demand)  (inventory /
newsvendor)
ie_newsvendor_uniform               Newsvendor order quantity (uniform demand)  (inventory /
newsvendor)
… 12 more
```

2.3 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ industrial-engineering list
ie_abc_classification                    ABC inventory classification
ie_acceptance_sampling                   Single sampling plan: OC, AOQ and ATI
ie_arl_shewhart                          Average run length of a Shewhart X-bar chart
ie_availability                          Inherent availability from MTBF and MTTR
ie_break_even                            Break-even point
ie_c_chart_limits                        c chart control limits
ie_capacity_utilization                  Capacity utilisation and efficiency
ie_center_of_gravity                     Centre-of-gravity location
… 86 more
```

2.4 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ industrial-engineering info ie_eoq
Economic order quantity (EOQ)
  Q* = sqrt(2 D S / H); TC = D S / Q + H Q / 2 = sqrt(2 D S H)
  in   demand   demand per year (units) []
  in   order_cost fixed cost per order or setup []
  in   holding_cost holding cost per unit per year []
  out  Q        economic order quantity Q* []
  out  orders_per_year orders per year D/Q* []
  out  cycle_time time between orders Q*/D (years) []
  out  ordering_cost annual ordering cost DS/Q* []
  out  holding_cost_per_year annual holding cost HQ*/2 []
  out  total_cost annual ordering plus holding cost []
```

2.5 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ industrial-engineering solve ie_eoq demand=1200.0 order_cost=100.0 holding_cost=6.0
Q = 200
orders_per_year = 6
cycle_time = 0.1666666667
ordering_cost = 600
holding_cost_per_year = 600
total_cost = 1200
  by: Economic order quantity (EOQ)  (Q* = sqrt(2 D S / H); TC = D S / Q + H Q / 2 = sqrt(2 D S H))
  demand = 1200
  order_cost = 100
  holding_cost = 6
  assumes: demand > 0; order_cost > 0; holding_cost > 0 (per unit per year)
```

2.6 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ industrial-engineering solve ie_eoq demand=1200.0 order_cost=100.0 holding_cost=6.0 --json
{"outcome":"SOLVED","message":"","value":200.0,"unit":"","solved_for":"Q","si":[["demand",1200.0],
["order_cost",100.0],["holding_cost",6.0]],"values":[["Q",200.0,""],["orders_per_year",6.0,""],
["cycle_time",0.166666666667,""],["ordering_cost",600.0,""],["holding_cost_per_year",600.0,""],
["total_cost",1200.0,""]],"target":"Q","phrase":"economic order quantity Q*","assumptions":["demand
> 0","order_cost > 0","holding_cost > 0 (per unit per year)"],"name":"Economic order quantity
(EOQ)","equation":"Q* = sqrt(2 D S / H); TC = D S / Q + H Q / 2 = sqrt(2 D S H)"}
```

2.7 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ industrial-engineering solve ie_eoq demand=1200.0 order_cost=100.0
NEEDS_INPUT: ie_eoq needs holding_cost (holding cost per unit per year). Example: solve('ie_eoq',
demand=1200, order_cost=100, holding_cost=6).
```

2.8 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | industrial-engineering batch

`cases.jsonl`:

```
{"key": "ie_eoq", "inputs": {"demand": 1200.0, "order_cost": 100.0, "holding_cost": 6.0}}
{"key": "ie_mmc_queue", "inputs": {"arrival_rate": 10.0, "service_rate": 4.0, "servers": "3"}}
```

```
$ industrial-engineering batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":200.0,"unit":"","solved_for":"Q","si":[["demand",1200.0],
["order_cost",100.0],["holding_cost",6.0]],"values":[["Q",200.0,""],["orders_per_year",6.0,""],
["cycle_time",0.166666666667,""],["ordering_cost",600.0,""],["holding_cost_per_year",600.0,""],
["total_cost",1200.0,""]],"target":"Q","phrase":"economic order quantity Q*","assumptions":["demand
> 0","order_cost > 0","holding_cost > 0 (per unit per year)"],"name":"Economic order quantity
(EOQ)","equation":"Q* = sqrt(2 D S / H); TC = D S / Q + H Q / 2 = sqrt(2 D S H)"}
{"outcome":"SOLVED","message":"","value":3.51123595506,"unit":"","solved_for":"Lq","si":
[["arrival_rate",10.0],["service_rate",4.0]],"values":[["utilization",0.833333333333,""],
["p_wait",0.702247191011,""],["Lq",3.51123595506,""],["Wq",0.351123595506,""],
["W",0.601123595506,""],["L",6.01123595506,""],
["p0",0.0449438202247,""]],"target":"Lq","phrase":"rho = lambda/(c mu)","assumptions":
["arrival_rate < servers x service_rate","1 <= servers <= 500"],"name":"M/M/c queue (Erlang
C)","equation":"a = lambda/mu, P0 = [sum_{k<c} a^k/k! + a^c/(c!(1-rho))]^-1, C = a^c P0/(c!(1-
rho)), Lq = C rho/(1-rho)"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `industrial-engineering info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI

kind	example	how it is read
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`industrial-engineering solve ... --json` prints one answer as one line of JSON. `industrial-engineering batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | industrial-engineering batch`. `industrial-engineering catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["industrial-engineering", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`industrial-engineering figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `industrial_engineering_ai999x.wasm` (https://www.ai999x.com/compute/rust/industrial_engineering_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("industrial_engineering_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `industrial-engineering info <id>` shows the same inputs and outputs in the terminal.

Break-even analysis (2)

Break-even point, `ie_break_even`

Volume (and revenue) at which revenue equals fixed plus variable cost, and the volume for a target profit.

$$Q_{BE} = \frac{F}{p - v}, \quad R_{BE} = p\,Q_{BE}$$

symbol		meaning	SI unit
`fixed_cost`	in	fixed cost per period	,
`price`	in	selling price per unit	,
`variable_cost`	in	variable cost per unit	,

symbol		meaning	SI unit
`target_profit`	in	target profit (optional) (optional)	,
`break_even_units`	out	F / (p - v)	,
`break_even_revenue`	out	p F / (p - v)	,
`contribution_margin`	out	p - v	,
`contribution_margin_ratio`	out	(p - v)/p	,
`units_for_target_profit`	out	(F + target)/(p - v)	,

Applies when: fixed_cost >= 0; price > variable_cost >= 0.

```
$ industrial-engineering solve ie_break_even fixed_cost=10000 price=25 variable_cost=15
break_even_units = 1000
break_even_revenue = 25000
contribution_margin = 10
contribution_margin_ratio = 0.4
  by: Break-even point  (Q_BE = F / (p - v))
  fixed_cost = 10000
  price = 25
  variable_cost = 15
  assumes: fixed_cost >= 0; price > variable_cost >= 0
```

What the worked example means: The break-even volume is 1000 units, generating revenue of 25000, with a contribution margin of 10.

Multiproduct break-even (sales-mix weighted), `ie_multiproduct_break_even`

Break-even revenue of several products sold in a fixed revenue mix: F / sum (1 - v_i/p_i) w_i.

$$BEP_{\$} = \frac{F}{\sum_i \left(1 - \frac{V_i}{P_i}\right) W_i}$$

symbol		meaning	SI unit
`fixed_cost`	in	fixed cost per period	,
`prices`	in	selling price of each product	,
`variable_costs`	in	variable cost per unit of each product	,
`sales_mix`	in	share of revenue of each product (sum 1)	,
`break_even_revenue`	out	F / sum (1 - v_i/p_i) w_i	,
`weighted_contribution_ratio`	out	sum (1 - v_i/p_i) w_i	,
`revenue_by_product`	out	break-even revenue of each product	,
`units_by_product`	out	break-even units of each product	,

Applies when: fixed_cost >= 0; prices > 0; mix fractions of revenue adding to 1.

```
$ industrial-engineering solve ie_multiproduct_break_even fixed_cost=3000 "prices=[2.95, 1.25,
0.8]" "variable_costs=[1.25, 0.3, 0.2]" "sales_mix=[0.5, 0.3, 0.2]"
break_even_revenue = 4503.587604
weighted_contribution_ratio = 0.6661355932
revenue_by_product = [2251.79380184,1351.07628111,900.717520737]
units_by_product = [763.319932828,1080.86102488,1125.89690092]
  by: Multiproduct break-even (sales-mix weighted)  (BEP($) = F / sum [(1 - V_i/P_i) W_i])
  fixed_cost = 3000
  assumes: fixed_cost >= 0; prices > 0; mix fractions of revenue adding to 1
```

What the worked example means: This calculation determines the total sales revenue required to cover all fixed costs when multiple products are sold in a specific mix. By dividing the period's fixed cost, which is 3000, by the sum of each product's contribution margin percentage weighted by its share of total revenue, this gives the overall break-even point. The resulting weighted contribution ratio is 0.666136, leading to a necessary total revenue of 4503.59. This figure represents the exact sales level where total revenues equal total costs, ensuring neither profit nor loss occurs across the entire product portfolio.

Facility location (5)

Centre-of-gravity location, `ie_center_of_gravity`

Location of a facility at the load-weighted average of the demand points' coordinates.

$$x^* = \frac{\sum w_i x_i}{\sum w_i}, \quad y^* = \frac{\sum w_i y_i}{\sum w_i}$$

symbol		meaning	SI unit
`x`	in	x coordinate of each demand point	,
`y`	in	y coordinate of each demand point	,
`weights`	in	load or volume of each point	,
`x`	out	sum w x / sum w	,
`y`	out	sum w y / sum w	,
`total_weight`	out	sum of loads	,

Applies when: equal lengths; weights >= 0, not all 0.

```
$ industrial-engineering solve ie_center_of_gravity "x=[30, 90, 130, 60]" "y=[120, 110, 130, 40]"
"weights=[2000, 1000, 1000, 2000]"
x = 66.66666667
y = 93.33333333
total_weight = 6000
  by: Centre-of-gravity location  (x* = sum w_i x_i / sum w_i, y* = sum w_i y_i / sum w_i)
  assumes: equal lengths; weights >= 0, not all 0
```

What the worked example means: This calculation determines the centre-of-gravity location for a new facility by finding the weighted average of existing demand points. The coordinates are derived from the inputs x = [30, 90, 130, 60], y = [120, 110, 130, 40], and weights = [2000, 1000, 1000, 2000]. The resulting position, given as x* = 66.6667 and y* = 93.3333, represents the point that minimizes the weighted distance to all demand locations, assuming transportation costs are proportional to load volume and distance.

Factor-rating method for site selection, `ie_factor_rating`

Weighted score of each candidate site (sum of factor weight x score) and the best site.

$$S_i = \sum_j w_j \, s_{ij}$$

symbol		meaning	SI unit
`weights`	in	weight of each factor	,
`scores`	in	matrix: one row per site, one score per factor	,
`weighted_scores`	out	weighted score of each site	,
`best_site`	out	site with the highest score	,
`best_score`	out	highest weighted score	,

Applies when: weights >= 0; one score per factor in every row; ties go to the lower site number.

```
$ industrial-engineering solve ie_factor_rating "weights=[0.25, 0.45, 0.3]" "scores=[[70, 60, 90],
[85, 80, 60]]"
weighted_scores = [71.5,75.25]
best_site = 2
best_score = 75.25
  by: Factor-rating method for site selection  (score_i = sum_j w_j s_ij)
  assumes: weights >= 0; one score per factor in every row; ties go to the lower site number
```

What the worked example means: This calculation determines the suitability of candidate sites by computing a weighted score for each. The process multiplies the importance weight assigned to each specific factor by the performance score given to that factor for a particular site, then sums these products to find the total weighted score for every site. The result shows that site 2 achieved the highest overall rating of 75.25. This indicates that, according to the criteria defined in the inputs, this location offers the best balance of factors relative to their assigned importance levels compared to all other candidates.

Locational cost-volume (break-even) analysis, `ie_location_cost_volume`

Total annual cost of each candidate site at an expected volume (fixed + variable x volume) and the cheapest site.

$$TC_i(Q) = F_i + v_i \, Q$$

symbol		meaning	SI unit
`fixed_costs`	in	annual fixed cost of each site	,
`variable_costs`	in	variable cost per unit at each site	,
`volume`	in	expected annual volume	,
`total_costs`	out	F_i + v_i Q for each site	,
`best_site`	out	cheapest site	,
`best_cost`	out	its total cost	,

Applies when: costs >= 0; volume >= 0; ties go to the lower site number.

```
$ industrial-engineering solve ie_location_cost_volume "fixed_costs=[30000, 60000, 110000]"
"variable_costs=[75, 45, 25]" volume=2000
total_costs = [180000.0,150000.0,160000.0]
best_site = 2
best_cost = 150000
  by: Locational cost-volume (break-even) analysis  (TC_i = F_i + v_i Q; the minimum wins)
  volume = 2000
  assumes: costs >= 0; volume >= 0; ties go to the lower site number
```

What the worked example means: Locational cost-volume analysis determines the most economical site by calculating the total annual cost for each candidate location using the formula where total cost equals fixed cost plus variable cost per unit multiplied by the expected volume. The system computes [180000, 150000, 160000] for every site based on their respective fixed and variable expenses and the input volume 2000. The site with the lowest total cost is selected as the best site, which in this case is 2, resulting in a minimum annual cost of 150000. This method assumes that fixed costs remain constant regardless of production volume within the relevant range and that variable costs scale linearly with output.

Minimum rectilinear-distance location (weighted median), `ie_rectilinear_location`

Location minimising the sum of load x rectilinear (city-block) distance: the weighted median of x and of y.

$$\min_{x,y} \sum_{i} w_i \left(|x - x_i| + |y - y_i| \right)$$

symbol		meaning	SI unit
`x`	in	x coordinate of each demand point	,
`y`	in	y coordinate of each demand point	,
`weights`	in	load or volume of each point	,
`x`	out	weighted median of x	,
`y`	out	weighted median of y	,
`total_weighted_distance`	out	sum w_i (|x - x_i| + |y - y_i|)	,

Applies when: weights >= 0, not all 0; refused when the optimum is a whole interval (cumulative weight exactly half).

```
$ industrial-engineering solve ie_rectilinear_location "x=[3, 8, 5, 1]" "y=[2, 6, 9, 4]" "weights=
[5, 2, 3, 1]"
x = 3
y = 4
total_weighted_distance = 47
  by: Minimum rectilinear-distance location (weighted median)  (sort by coordinate; the first point
where cumulative weight reaches half the total)
  assumes: weights >= 0, not all 0; refused when the optimum is a whole interval (cumulative weight
exactly half)
```

What the worked example means: This calculation determines the facility location that minimizes the sum of weighted city-block distances to all demand points. By sorting the coordinates and identifying the weighted

median, this gives the x-coordinate 3 and y-coordinate 4. The resulting minimum total cost, representing the aggregate load multiplied by distance, is 47. This method ensures that moving the facility in any direction would increase the overall transportation burden.

Minimum weighted-distance location (Weber problem), `ie_weber_location`

Location that minimises the sum of load x straight-line distance to the demand points (Weiszfeld's algorithm; a demand point when it is optimal).

$$\min_{x,y} \sum_i w_i \sqrt{(x - x_i)^2 + (y - y_i)^2}$$

symbol		meaning	SI unit
`x`	in	x coordinate of each demand point	,
`y`	in	y coordinate of each demand point	,
`weights`	in	load or volume of each point	,
`x`	out	optimal x	,
`y`	out	optimal y	,
`total_weighted_distance`	out	sum w_i distance_i at the optimum	,

Applies when: at least 2 distinct points; weights >= 0, not all 0; Euclidean distance.

```
$ industrial-engineering solve ie_weber_location "x=[0, 10, 5, 2]" "y=[0, 0, 8, 6]" "weights=[1, 1,
1, 1]"
x = 3.191489362
y = 5.106382979
total_weighted_distance = 19.43398113
  by: Minimum weighted-distance location (Weber problem)  (Weiszfeld: x <- sum (w_i x_i / d_i) /
sum (w_i / d_i) until it settles; a demand point i is optimal when |sum_{j != i} w_j (p_i -
p_j)/d_ij| <= w_i)
  assumes: at least 2 distinct points; weights >= 0, not all 0; Euclidean distance
```

What the worked example means: This calculation determines the facility location that minimizes the sum of weighted Euclidean distances to all demand points. Using the Weiszfeld algorithm, it iteratively refines the coordinates until convergence or identifies a demand point as optimal if the condition holds. The resulting position at 3.19149 and 5.10638 represents the geometric median weighted by load, producing a minimum aggregate transportation cost metric of 19.434.

Forecasting (9)

Associative forecast by simple linear regression, `ie_regression_forecast`

Least-squares line y = a + b x of demand on a driver, its correlation and standard error of estimate, and the forecast at a new x.

$$\hat{y} = a + bx, \quad S_{y,x} = \sqrt{\frac{\sum (y - \hat{y})^2}{n - 2}}$$

symbol		meaning	SI unit
`x`	in	values of the driver (independent variable)	,
`y`	in	values of demand (dependent variable)	,
`x_new`	in	driver value to forecast at (optional) (optional)	,
`intercept`	out	a	,
`slope`	out	b	,
`r`	out	correlation coefficient	,
`r_squared`	out	coefficient of determination	,
`standard_error`	out	standard error of the estimate sqrt(SSE/(n - 2))	,
`forecast`	out	a + b x_new	,

Applies when: at least 3 pairs; x and y vary.

```
$ industrial-engineering solve ie_regression_forecast "x=[1, 3, 4, 2, 1, 7]" "y=[2, 3, 2.5, 2, 2,
3.5]" x_new=6
intercept = 1.75
slope = 0.25
r = 0.9013878189
r_squared = 0.8125
standard_error = 0.3061862178
forecast = 3.25
  by: Associative forecast by simple linear regression  (ordinary least squares; S_yx = sqrt(sum (y
- y_hat)^2 / (n - 2)))
  x_new = 6
  assumes: at least 3 pairs; x and y vary
```

What the worked example means: Ordinary least squares fits a straight line to relate demand to its driver. The computed slope is 0.25 and the intercept is 1.75. This model explains 0.8125 of the variation in demand, indicated by a correlation coefficient of 0.901388. The standard error of the estimate is 0.306186, reflecting the typical distance between observed data points and the fitted line. Using this relationship, the forecasted demand for a new driver value of 6 is 3.25.

Forecast accuracy: MAD, MSE, RMSE, MAPE, bias, tracking signal, `ie_forecast_errors`

Standard accuracy measures of a forecast against actual values (error = actual - forecast).

$$MAD = \frac{\sum |e_t|}{n},\ MSE = \frac{\sum e_t^2}{n},\ MAPE = \frac{100}{n}\sum \left|\frac{e_t}{A_t}\right|,\ TS = \frac{\sum e_t}{MAD}$$

symbol		meaning	SI unit
`actual`	in	actual values	,
`forecast`	in	forecast values (same periods)	,
`mad`	out	mean absolute deviation	,
`mse`	out	mean squared error	,
`rmse`	out	root mean squared error	,
`mape`	out	mean absolute percentage error, in percent (when no actual is 0)	,

symbol		meaning	SI unit
`bias`	out	mean error (positive: forecasts too low)	,
`running_sum_of_errors`	out	sum of errors	,
`tracking_signal`	out	running sum of errors / MAD	,

Applies when: equal lengths; error = actual - forecast; MAPE only if no actual value is 0.

```
$ industrial-engineering solve ie_forecast_errors "actual=[100, 110, 120, 130]" "forecast=[105,
108, 125, 126]"
mad = 4
mse = 17.5
rmse = 4.183300133
bias = -1
running_sum_of_errors = -4
mape = 3.51544289
tracking_signal = -1
  by: Forecast accuracy: MAD, MSE, RMSE, MAPE, bias, tracking signal  (e_t = A_t - F_t; MAD = mean
|e|; MSE = mean e^2; MAPE = 100 mean |e/A|; TS = sum e / MAD)
  assumes: equal lengths; error = actual - forecast; MAPE only if no actual value is 0
```

What the worked example means: This calculation evaluates the accuracy of a demand or supply forecast by comparing actual values against predicted ones. The mean absolute deviation, reported as 4, represents the average magnitude of errors without considering direction, while the root mean squared error, shown as 4.1833, penalizes larger errors more heavily. The bias, indicated by -1, reveals whether the forecasts are consistently too high or too low, and the tracking signal of -1 helps identify if this bias is significant relative to the typical error size.

Holt's linear trend (double exponential smoothing), `ie_holt_trend`

Trend-adjusted exponential smoothing: level and trend updated each period; the forecast h periods ahead is level + h x trend.

$$L_t = \alpha A_t + (1-\alpha)(L_{t-1} + T_{t-1}), \quad T_t = \beta(L_t - L_{t-1}) + (1-\beta)T_{t-1}$$

symbol		meaning	SI unit
`data`	in	observations in time order (oldest first)	,
`alpha`	in	level smoothing constant	,
`beta`	in	trend smoothing constant	,
`initial_level`	in	level before the first observation	,
`initial_trend`	in	trend before the first observation	,
`horizon`	in	periods ahead (default 1) (optional)	,
`forecast`	out	level + horizon x trend	,
`level`	out	final level	,
`trend`	out	final trend	,

Applies when: 0 < alpha, beta <= 1; 1 <= horizon <= 1000.

```
$ industrial-engineering solve ie_holt_trend "data=[12, 17, 20, 19, 24, 27]" alpha=0.2 beta=0.4
initial_level=11 initial_trend=2
forecast = 27.85924608
level = 25.31331578
trend = 2.545930297
  by: Holt's linear trend (double exponential smoothing)  (L_t = alpha A_t + (1 - alpha)(L_(t-1) +
T_(t-1)); T_t = beta (L_t - L_(t-1)) + (1 - beta) T_(t-1); F_(t+h) = L_t + h T_t)
  alpha = 0.2
  beta = 0.4
  initial_level = 11
  initial_trend = 2
  assumes: 0 < alpha, beta <= 1; 1 <= horizon <= 1000
```

In the worked example, data = [12, 17, 20, 19, 24, 27], alpha = 0.2, beta = 0.4, initial_level = 11, initial_trend = 2 go through the equation above and give forecast = 27.85924608, level = 25.31331578, trend = 2.545930297. It applies only when 0 < alpha, beta <= 1; 1 <= horizon <= 1000: outside that it says what is wrong instead of returning a number.

Linear trend projection (least squares), `ie_linear_trend_forecast`

Least-squares trend line y = a + b t through the data at periods t = 1, ..., n, projected h periods ahead.

$$b = \frac{\sum (t - \bar{t})(y - \bar{y})}{\sum (t - \bar{t})^2}, \quad a = \bar{y} - b\bar{t}$$

symbol		meaning	SI unit
`data`	in	observations in time order (oldest first)	,
`horizon`	in	periods beyond the last observation (default 1) (optional)	,
`forecast`	out	a + b (n + h)	,
`intercept`	out	a	,
`slope`	out	b, change per period	,

Applies when: at least 2 observations; periods numbered 1, 2, ..., n.

```
$ industrial-engineering solve ie_linear_trend_forecast "data=[74, 79, 80, 90, 105, 142, 122]"
forecast = 141
intercept = 56.71428571
slope = 10.53571429
  by: Linear trend projection (least squares)  (b = sum (t - t_bar)(y - y_bar) / sum (t - t_bar)^2,
a = y_bar - b t_bar)
  assumes: at least 2 observations; periods numbered 1, 2, ..., n
```

What the worked example means: This calculation determines the linear trend line that best fits the historical data using the least squares method. It computes an intercept and a slope to describe how the metric changes over time, then projects this pattern forward by the specified horizon. The resulting forecast of 141 reflects the underlying directional movement captured by the slope of 10.5357.

Moving average forecast, `ie_moving_average_forecast`

Next-period forecast as the average of the last n observations (and the forecast each period would have had).

$$F_{t+1} = \frac{1}{n} \sum_{i=0}^{n-1} A_{t-i}$$

symbol		meaning	SI unit
`data`	in	observations in time order (oldest first)	,
`periods`	in	number of periods averaged n	,
`forecast`	out	forecast for the next period	,
`forecasts`	out	moving averages: forecast for periods n+1, ..., (last period + 1)	,

Applies when: 1 <= periods <= number of observations.

```
$ industrial-engineering solve ie_moving_average_forecast "data=[120, 135, 128, 142, 150, 147]"
periods=3
forecast = 146.3333333
forecasts = [127.666666667,135.0,140.0,146.333333333]
  by: Moving average forecast  (F_(t+1) = (A_t + ... + A_(t-n+1)) / n)
  assumes: 1 <= periods <= number of observations
```

What the worked example means: This calculation determines the moving average forecast for the upcoming period by averaging the most recent historical data points. The method takes the sum of the last 3 observations from the input data and divides that total by the number of periods to produce a single value, 146.333. This result represents the expected demand or level for the next time step, assuming that recent history is the best predictor of the immediate future. It smooths out short-term fluctuations to reveal the underlying trend in the series.

Seasonal indices (average method), `ie_seasonal_index`

Seasonal index of each season = average demand in that season / average demand over all seasons; with an expected annual total, the seasonal forecasts.

$$S_k = \frac{\bar{A}_k}{\bar{A}}, \quad F_k = \frac{T}{s} S_k$$

symbol		meaning	SI unit
`data`	in	matrix: one row per year, one column per season (month, quarter)	,
`next_year_total`	in	expected total demand next year (optional) (optional)	,
`indices`	out	seasonal index of each season	,
`season_averages`	out	average of each season	,
`overall_average`	out	average over all seasons and years	,
`forecasts`	out	next-year forecast of each season (annual total / seasons x index)	,

Applies when: rows of equal length (at least 2 seasons); positive average.

```
$ industrial-engineering solve ie_seasonal_index "data=[[80, 85, 105, 110], [70, 85, 85, 120], [80,
93, 82, 98]]" next_year_total=1200
indices = [0.841720036597,0.962488563586,0.995425434584,1.20036596523]
season_averages = [76.6666666667,87.6666666667,90.6666666667,109.333333333]
overall_average = 91.08333333
forecasts = [252.516010979,288.746569076,298.627630375,360.10978957]
  by: Seasonal indices (average method)  (index_k = mean of season k / overall mean; forecast_k =
(annual total / s) index_k)
  next_year_total = 1200
  assumes: rows of equal length (at least 2 seasons); positive average
```

What the worked example means: This calculation determines the seasonal pattern of demand by comparing the average value in each specific season against the global average across all periods. The resulting seasonal indices, shown as [0.84172, 0.962489, 0.995425, 1.20037], indicate how much higher or lower a particular season's demand is compared to the typical period. If an expected annual total is provided, these indices are used to generate forecasts for the next year, displayed as [252.516, 288.747, 298.628, 360.11], effectively distributing the total demand according to historical seasonal trends.

Simple exponential smoothing forecast, `ie_exponential_smoothing`

Exponentially smoothed forecasts F_(t+1) = F_t + alpha (A_t - F_t), starting from a given first forecast (default: the first observation).

$$F_{t+1} = F_t + \alpha\,(A_t - F_t)$$

symbol		meaning	SI unit
`data`	in	observations in time order (oldest first)	,
`alpha`	in	smoothing constant (0 < alpha <= 1)	,
`initial_forecast`	in	forecast for period 1 (default: the first observation) (optional)	,
`forecast`	out	forecast for the next period	,
`forecasts`	out	forecasts for periods 1, ..., n + 1	,

Applies when: 0 < alpha <= 1.

```
$ industrial-engineering solve ie_exponential_smoothing "data=[142, 153, 136, 150]" alpha=0.2
initial_forecast=140
forecast = 143.2288
forecasts = [140.0,140.4,142.92,141.536,143.2288]
  by: Simple exponential smoothing forecast  (F_1 given (or A_1); F_(t+1) = alpha A_t + (1 - alpha)
F_t)
  alpha = 0.2
  initial_forecast = 140
  assumes: 0 < alpha <= 1
```

What the worked example means: This calculation produces a smoothed forecast for the upcoming period, denoted as 143.229. The result relies on the smoothing constant 0.2, which determines how much weight is given to the most recent observation versus previous forecasts. By applying this exponential weighting scheme, the method adapts to recent changes in demand while filtering out noise, providing a balanced prediction based on the historical series provided in [142, 153, 136, 150].

Tracking signal over time, `ie_tracking_signal`

Tracking signal each period (running sum of errors / running MAD) and the first period it leaves the +/- limit.

$$TS_t = \frac{\sum_{i \le t} e_i}{MAD_t}$$

symbol		meaning	SI unit
`actual`	in	actual values	,
`forecast`	in	forecast values (same periods)	,
`limit`	in	control limit in MADs (default 4) (optional)	,
`final_signal`	out	tracking signal in the last period	,
`signals`	out	tracking signal of each period	,
`first_out_of_control`	out	first period with |TS| > limit (0 = never)	,

Applies when: equal lengths; running MAD uses all periods so far.

```
$ industrial-engineering solve ie_tracking_signal "actual=[90, 95, 115, 100, 125, 140]" "forecast=
[100, 100, 100, 110, 110, 110]"
final_signal = 2.470588235
signals = [-1.0,-2.0,0.0,-1.0,0.454545454545,2.47058823529]
first_out_of_control = 0
  by: Tracking signal over time  (TS_t = (sum_{i<=t} e_i) / (sum_{i<=t} |e_i| / t))
  assumes: equal lengths; running MAD uses all periods so far
```

What the worked example means: The tracking signal reached 2.47059 in the last period, with the first out-of-control period being 0.

Weighted moving average forecast, `ie_weighted_moving_average`

Next-period forecast as a weighted average of the most recent observations (weights listed oldest to newest, divided by their sum).

$$F_{t+1} = \frac{\sum_{i=1}^{k} w_i A_{t-k+i}}{\sum w_i}$$

symbol		meaning	SI unit
`data`	in	observations in time order (oldest first)	,
`weights`	in	weights for the last k observations, oldest first	,
`forecast`	out	sum w_i A_i / sum w_i	,
`weight_sum`	out	sum of the weights	,

Applies when: weights >= 0, not all 0; at least as many observations as weights; the last weight applies to the most recent observation.

```
$ industrial-engineering solve ie_weighted_moving_average "data=[100, 110, 105, 120]" "weights=[1,
2, 3]"
forecast = 113.3333333
weight_sum = 6
  by: Weighted moving average forecast  (F = sum w_i A_i / sum w_i over the last k observations)
  assumes: weights >= 0, not all 0; at least as many observations as weights; the last weight
applies to the most recent observation
```

What the worked example means: This calculation determines the next period forecast by taking a weighted average of the most recent observations. The weights are applied to the data in time order, with the last weight corresponding to the newest observation. The sum of these weighted values is divided by the total weight sum of 6 to produce the final forecast value of 113.333.

Inventory (18)

(R, Q) reorder point for a target fill rate, `ie_rq_fill_rate`

Reorder point of a continuous-review (R, Q) policy that meets a fill rate (fraction of demand met from stock) with normal lead-time demand.

$$\sigma_L \, L(z) = (1 - \beta)\, Q, \quad R = \mu_L + z\,\sigma_L$$

symbol		meaning	SI unit
`mean_ltd`	in	mean demand during the lead time	,
`sd_ltd`	in	standard deviation of lead-time demand	,
`order_quantity`	in	order quantity Q	,
`fill_rate`	in	target fill rate (decimal)	,
`reorder_point`	out	reorder point R	,
`safety_stock`	out	safety stock R - mu	,
`z`	out	service factor	,
`expected_shortage_per_cycle`	out	expected units short per cycle (1 - beta) Q	,
`cycle_service_level`	out	probability of no stock-out per cycle	,

Applies when: sd_ltd, order_quantity > 0; 0 < fill_rate < 1; lead-time demand normal.

```
$ industrial-engineering solve ie_rq_fill_rate mean_ltd=500 sd_ltd=100 order_quantity=1000
fill_rate=0.99
reorder_point = 590.2346348
safety_stock = 90.23463475
z = 0.9023463475
expected_shortage_per_cycle = 10
cycle_service_level = 0.8165635438
  by: (R, Q) reorder point for a target fill rate  (solve sigma L(z) = (1 - beta) Q for z (L the
standard normal loss function); R = mu + z sigma)
  mean_ltd = 500
  sd_ltd = 100
  order_quantity = 1000
  fill_rate = 0.99
  assumes: sd_ltd, order_quantity > 0; 0 < fill_rate < 1; lead-time demand normal
```

What the worked example means: This calculation determines the inventory level at which a new order should be placed to achieve a specific service goal. By solving for the service factor z that satisfies the equation where the standard deviation times the loss function equals the expected shortage per cycle, this gives the safety stock required. The final reorder point is the sum of the average demand during lead time and this safety buffer. For example, with an order quantity of 1000 and a desired fill rate of 0.99, the resulting reorder point is 590.235, implying a safety stock of 90.2346. This ensures that the probability of stocking out in any given cycle is managed effectively.

ABC inventory classification, `ie_abc_classification`

Classify items by annual usage value: A items make up the first ~80 % of value, B the next ~15 %, C the rest.

$$\text{class}(k) = A \text{ if } \frac{\sum_{j \prec k} v_j}{\sum v} < 0.8,\ B \text{ if } < 0.95,\ C$$

symbol		meaning	SI unit
`annual_values`	in	annual usage value of each item (annual demand x unit cost)	,
`a_share`	in	cumulative value share closing class A (default 80%) (optional)	,
`b_share`	in	cumulative value share closing class B (default 95%) (optional)	,
`classes`	out	class of each item, in input order	,
`count_a`	out	number of A items	,
`count_b`	out	number of B items	,
`count_c`	out	number of C items	,
`share_a`	out	share of total value in class A	,
`share_b`	out	share of total value in class B	,

Applies when: values >= 0, not all 0; 0 < a_share < b_share < 1; items ranked by value (ties by item number); an item belongs to A while the value ranked above it is below a_share of the total (so the item crossing the threshold is still A), likewise B.

```
$ industrial-engineering solve ie_abc_classification "annual_values=[90000, 15000, 4000, 60000,
2500, 12000, 1500, 800, 30000, 6000]"
classes = ["A","B","C","A","C","B","C","C","A","B"]
count_a = 3
count_b = 3
count_c = 4
share_a = 0.8115419297
share_b = 0.1487826871
  by: ABC inventory classification  (rank by value; cumulative share before each item decides its
class)
  assumes: values >= 0, not all 0; 0 < a_share < b_share < 1; items ranked by value (ties by item
number); an item belongs to A while the value ranked above it is below a_share of the total (so the
item crossing the threshold is still A), likewise B
```

What the worked example means: Items are classified into A, B, and C groups based on annual value, with 3 items in class A.

Annual inventory cost of an order quantity, `ie_inventory_total_cost`

Annual ordering, holding and purchase cost of ordering a given quantity Q each time (constant demand).

$$TC(Q) = \frac{D}{Q}S + \frac{Q}{2}H + Dc$$

symbol		meaning	SI unit
`demand`	in	demand per year (units)	,
`order_cost`	in	fixed cost per order or setup	,
`holding_cost`	in	holding cost per unit per year	,
`order_quantity`	in	quantity ordered each time	,
`unit_cost`	in	purchase price per unit (optional, default 0) (optional)	,
`total_cost`	out	annual total cost DS/Q + HQ/2 + Dc	,
`ordering_cost`	out	annual ordering cost	,
`holding_cost_per_year`	out	annual holding cost	,
`purchase_cost`	out	annual purchase cost	,
`average_inventory`	out	average inventory Q/2	,

Applies when: demand, order_cost, holding_cost, order_quantity > 0; unit_cost >= 0.

```
$ industrial-engineering solve ie_inventory_total_cost demand=1200 order_cost=100 holding_cost=6
order_quantity=300
total_cost = 1300
ordering_cost = 400
holding_cost_per_year = 900
purchase_cost = 0
average_inventory = 150
  by: Annual inventory cost of an order quantity  (TC = D S / Q + H Q / 2 + D c)
  demand = 1200
  order_cost = 100
  holding_cost = 6
  order_quantity = 300
  assumes: demand, order_cost, holding_cost, order_quantity > 0; unit_cost >= 0
```

What the worked example means: This calculation determines the yearly inventory cost for a fixed order quantity. It sums three components: the annual ordering cost of 400, derived from demand and setup fees; the annual holding cost of 900, based on average inventory levels; and the annual purchase cost of 0. The resulting total annual cost is 1300, which helps evaluate the financial impact of choosing specific batch sizes under constant demand conditions.

Economic order quantity (EOQ), `ie_eoq`

The order quantity that minimises annual ordering plus holding cost for constant demand: Q* = sqrt(2DS/H).

$$Q^* = \sqrt{\frac{2DS}{H}}, \quad TC^* = \sqrt{2DSH}$$

symbol		meaning	SI unit
`demand`	in	demand per year (units)	,
`order_cost`	in	fixed cost per order or setup	,
`holding_cost`	in	holding cost per unit per year	,
`Q`	out	economic order quantity Q*	,
`orders_per_year`	out	orders per year D/Q*	,
`cycle_time`	out	time between orders Q*/D (years)	,
`ordering_cost`	out	annual ordering cost DS/Q*	,
`holding_cost_per_year`	out	annual holding cost HQ*/2	,
`total_cost`	out	annual ordering plus holding cost	,

Applies when: demand > 0; order_cost > 0; holding_cost > 0 (per unit per year).

```
$ industrial-engineering solve ie_eoq demand=1200 order_cost=100 holding_cost=6
Q = 200
orders_per_year = 6
cycle_time = 0.1666666667
ordering_cost = 600
holding_cost_per_year = 600
total_cost = 1200
  by: Economic order quantity (EOQ)  (Q* = sqrt(2 D S / H); TC = D S / Q + H Q / 2 = sqrt(2 D S H))
  demand = 1200
  order_cost = 100
  holding_cost = 6
  assumes: demand > 0; order_cost > 0; holding_cost > 0 (per unit per year)
```

What the worked example means: This calculation determines the economic order quantity, which balances ordering and holding costs to minimize total inventory expenses. Given an annual demand of 1200 units, a fixed cost per order of 100, and a holding cost of 6 per unit per year, the model computes the ideal batch size. The result shows that ordering 200 units at a time leads to 6 orders per year, with each cycle lasting 0.166667 years. This strategy yields an annual ordering cost of 600 and an annual holding cost of 600, for a combined total cost of 1200.

Economic production quantity (EPQ), `ie_epq`

Production lot size minimising setup plus holding cost when stock builds up gradually at a finite production rate.

$$Q^* = \sqrt{\frac{2DS}{H\,(1 - D/P)}}$$

symbol		meaning	SI unit
`demand`	in	demand per year (units)	,
`production_rate`	in	production rate per year (same units as demand)	,
`order_cost`	in	fixed cost per order or setup	,
`holding_cost`	in	holding cost per unit per year	,
`Q`	out	economic production quantity	,
`max_inventory`	out	maximum inventory Q(1 - D/P)	,
`production_run_time`	out	length of a production run Q/P (years)	,
`cycle_time`	out	cycle Q/D (years)	,
`runs_per_year`	out	production runs per year	,
`total_cost`	out	annual setup plus holding cost	,

Applies when: all inputs > 0; production_rate > demand.

```
$ industrial-engineering solve ie_epq demand=12000 production_rate=48000 order_cost=400
holding_cost=3
Q = 2065.591118
max_inventory = 1549.193338
production_run_time = 0.04303314829
cycle_time = 0.1721325932
runs_per_year = 5.809475019
total_cost = 4647.580015
  by: Economic production quantity (EPQ)  (Q* = sqrt(2DS / (H (1 - D/P))), I_max = Q*(1 - D/P))
  demand = 12000
  production_rate = 48000
  order_cost = 400
  holding_cost = 3
  assumes: all inputs > 0; production_rate > demand
```

What the worked example means: This calculation determines the optimal batch size for production to minimize total costs. Using an annual demand of 12000 units, a production rate of 48000 units per year, a setup cost of 400, and a holding cost of 3, the model computes an economic production quantity of 2065.59. This results in a cycle time of 0.172133 years and a total annual cost of 4647.58.

EOQ with all-units quantity discounts, `ie_eoq_quantity_discount`

Best order quantity when the unit price falls at quantity breaks (all-units discount) and holding cost is a fraction of the unit price.

$$TC_k(Q) = \frac{D}{Q}S + \frac{Q}{2}\,i\,p_k + Dp_k$$

symbol		meaning	SI unit
`demand`	in	demand per year (units)	,
`order_cost`	in	fixed cost per order or setup	,
`holding_rate`	in	annual holding cost as a fraction of the unit price (decimal; 20% = 0.2)	,
`break_quantities`	in	minimum quantity of each price tier, starting with 0	,
`unit_prices`	in	unit price of each tier (falling)	,
`Q`	out	best order quantity	,
`unit_price`	out	unit price paid	,
`total_cost`	out	annual ordering + holding + purchase cost	,
`tier`	out	price tier chosen (1 = first)	,
`ordering_cost`	out	annual ordering cost	,
`holding_cost_per_year`	out	annual holding cost	,
`purchase_cost`	out	annual purchase cost	,

Applies when: break_quantities start at 0 and increase; unit_prices fall tier by tier; 0 < holding_rate < 10.

```
$ industrial-engineering solve ie_eoq_quantity_discount demand=5000 order_cost=49 holding_rate=0.2
"break_quantities=[0, 1000, 2000]" "unit_prices=[5, 4.8, 4.75]"
Q = 1000
unit_price = 4.8
total_cost = 24725
tier = 2
ordering_cost = 245
holding_cost_per_year = 480
purchase_cost = 24000
  by: EOQ with all-units quantity discounts  (for each tier: EOQ at its price, raised to the tier's
minimum if below it (skipped if above the next break); the lowest total cost D S/Q + i p Q/2 + D p
wins)
  demand = 5000
  order_cost = 49
  holding_rate = 0.2
  assumes: break_quantities start at 0 and increase; unit_prices fall tier by tier; 0 <
holding_rate < 10
```

What the worked example means: This calculation determines the most economical order size by evaluating each price tier. For every tier, it computes the classic economic order quantity and adjusts it to meet the minimum quantity requirement for that discount level. The algorithm then compares the total annual costs, which include ordering, holding, and purchase expenses, across all feasible quantities. The result shows that ordering 1000 units at 4.8 per unit minimizes the total annual cost to 24725, selecting tier 2.

EOQ with planned backorders, `ie_eoq_backorders`

Order quantity and maximum backorder level minimising ordering, holding and backorder cost when shortages are allowed and fully backordered.

$$Q^* = \sqrt{\frac{2DS}{H} \cdot \frac{H+B}{B}}, \quad b^* = Q^* \frac{H}{H+B}$$

symbol		meaning	SI unit
`demand`	in	demand per year (units)	,
`order_cost`	in	fixed cost per order or setup	,
`holding_cost`	in	holding cost per unit per year	,
`backorder_cost`	in	backorder (shortage) cost per unit per year	,
`Q`	out	optimal order quantity	,
`max_backorder`	out	maximum backorder level b*	,
`max_inventory`	out	maximum inventory Q* - b*	,
`total_cost`	out	minimum annual cost	,
`orders_per_year`	out	orders per year	,
`fraction_backordered_time`	out	fraction of time out of stock H/(H+B)	,

Applies when: all inputs > 0; every shortage is backordered.

```
$ industrial-engineering solve ie_eoq_backorders demand=1200 order_cost=100 holding_cost=6
backorder_cost=18
Q = 230.9401077
max_backorder = 57.73502692
max_inventory = 173.2050808
total_cost = 1039.230485
orders_per_year = 5.196152423
fraction_backordered_time = 0.25
  by: EOQ with planned backorders  (Q* = sqrt(2DS/H (H+B)/B), b* = Q* H/(H+B), TC* = sqrt(2DSH
B/(H+B)))
  demand = 1200
  order_cost = 100
  holding_cost = 6
  backorder_cost = 18
  assumes: all inputs > 0; every shortage is backordered
```

What the worked example means: This calculation determines the economic order quantity and planned backorder levels that minimize total annual inventory costs. Using an annual demand of 1200, an ordering cost of 100, a holding cost of 6, and a backorder cost of 18, the model computes an optimal order size of 230.94. The resulting minimum annual total cost is 1039.23, achieved by allowing up to 57.735 units to be on backorder at any given time.

Inventory turnover and days of supply, `ie_inventory_turnover`

How many times a year inventory turns over and how many days (weeks) of supply it holds.

$$\text{turns} = \frac{\text{COGS}}{\bar{I}}, \quad \text{days of supply} = \frac{365}{\text{turns}}$$

symbol		meaning	SI unit
`cost_of_goods_sold`	in	annual cost of goods sold	,
`average_inventory`	in	average inventory value (at cost)	,
`days_per_year`	in	days in the year used (default 365) (optional)	,
`turnover`	out	inventory turns per year COGS / average inventory	,
`days_of_supply`	out	days of supply	,
`weeks_of_supply`	out	weeks of supply	,

Applies when: all inputs > 0.

```
$ industrial-engineering solve ie_inventory_turnover cost_of_goods_sold=1200000
average_inventory=150000
turnover = 8
days_of_supply = 45.625
weeks_of_supply = 6.5
  by: Inventory turnover and days of supply  (turnover = COGS / average inventory; days = days per
year / turnover)
  cost_of_goods_sold = 1200000
  average_inventory = 150000
  assumes: all inputs > 0
```

What the worked example means: This calculation determines inventory efficiency by dividing the annual cost of goods sold by the average inventory value. The resulting turnover rate indicates how frequently stock is replaced within a year. A higher turnover generally suggests efficient management and strong sales velocity. The days of supply metric converts this rate into a time-based measure, showing exactly how long current inventory levels would last if sales continued at the historical average pace. For this scenario, with a cost of goods sold of 1.2e+06 and average inventory of 150000, the turnover is 8 and the days of supply are 45.625.

Newsvendor order quantity (discrete demand table), `ie_newsvendor_discrete`

Best single-period order quantity among the listed demand values, given their probabilities.

$$Q^* = \min\{d_k : F(d_k) \geq \frac{C_u}{C_u+C_o}\}$$

symbol		meaning	SI unit
`price`	in	selling price per unit	,
`cost`	in	purchase cost per unit	,
`demands`	in	possible demand values (increasing)	,
`probabilities`	in	probability of each demand value (sum 1)	,
`salvage`	in	salvage value per unsold unit (optional, default 0) (optional)	,
`Q`	out	optimal order quantity (one of the demand values)	,
`critical_ratio`	out	critical ratio	,
`expected_sales`	out	expected units sold	,
`expected_profit`	out	expected profit	,

Applies when: salvage < cost < price; demands increasing; probabilities add to 1; refused when two quantities are equally good.

```
$ industrial-engineering solve ie_newsvendor_discrete price=10 cost=4 "demands=[20, 30, 40, 50]"
"probabilities=[0.2, 0.3, 0.3, 0.2]"
Q = 40
critical_ratio = 0.6
expected_sales = 33
expected_profit = 170
  by: Newsvendor order quantity (discrete demand table)  (smallest demand value whose cumulative
probability exceeds Cu/(Cu+Co); expected profit sum p_k profit(Q, d_k))
  price = 10
  cost = 4
  assumes: salvage < cost < price; demands increasing; probabilities add to 1; refused when two
quantities are equally good
```

What the worked example means: This calculation determines the best single-period order quantity from a discrete list of demand values. It finds the smallest demand level where the cumulative probability meets or exceeds the critical ratio, which balances the cost of understocking against the cost of overstocking. The result shows that ordering 40 units maximizes expected value, producing an expected sales volume of 33 units and an expected profit of 170. This approach assumes demand probabilities sum to one and that salvage value is less than purchase cost, which is less than selling price.

Newsvendor order quantity (normal demand), `ie_newsvendor_normal`

Single-period order quantity that maximises expected profit when demand is normal: the critical ratio Cu/(Cu + Co).

$$F(Q^*) = \frac{C_u}{C_u + C_o}, \quad Q^* = \mu + z\sigma$$

symbol		meaning	SI unit
`price`	in	selling price per unit	,
`cost`	in	purchase cost per unit	,
`mean_demand`	in	mean demand per period (per day, say)	,
`sd_demand`	in	standard deviation of demand per period	,
`salvage`	in	salvage value per unsold unit (optional, default 0) (optional)	,
`Q`	out	optimal order quantity mu + z sigma	,
`critical_ratio`	out	critical ratio Cu/(Cu+Co)	,
`z`	out	z of the critical ratio	,
`expected_sales`	out	expected units sold	,
`expected_leftover`	out	expected units left over	,
`expected_shortage`	out	expected unmet demand	,
`expected_profit`	out	expected profit	,

Applies when: salvage < cost < price; sd_demand > 0; demand normal.

```
$ industrial-engineering solve ie_newsvendor_normal price=10 cost=6 mean_demand=500 sd_demand=100
salvage=2
Q = 500
critical_ratio = 0.5
z = 0
expected_sales = 460.105772
expected_leftover = 39.89422804
expected_shortage = 39.89422804
expected_profit = 1680.846176
  by: Newsvendor order quantity (normal demand)  (Cu = p - c, Co = c - v, CR = Cu/(Cu+Co), Q = mu +
Phi^-1(CR) sigma; E[sales] = mu - sigma L(z))
  price = 10
  cost = 6
  mean_demand = 500
  sd_demand = 100
  salvage = 2
  assumes: salvage < cost < price; sd_demand > 0; demand normal
```

What the worked example means: This calculation determines the best single-period inventory level to maximize expected profit. It uses the selling price, unit cost, and salvage value to find the critical fractile, which balances the cost of overstocking against the cost of understocking. With a mean demand of 500 and standard deviation of 100, the model selects an order quantity of 500. This results in an expected sales volume of 460.106 units and an expected leftover stock of 39.8942 units.

Newsvendor order quantity (Poisson demand), `ie_newsvendor_poisson`

Whole-unit single-period order quantity when demand is Poisson: the smallest Q with F(Q) >= critical ratio.

$$Q^* = \min\{Q : F(Q) \geq \frac{C_u}{C_u+C_o}\}$$

symbol		meaning	SI unit
`price`	in	selling price per unit	,
`cost`	in	purchase cost per unit	,
`mean_demand`	in	mean (Poisson) demand	,
`salvage`	in	salvage value per unsold unit (optional, default 0) (optional)	,
`Q`	out	optimal whole order quantity	,
`critical_ratio`	out	critical ratio	,
`expected_sales`	out	expected units sold	,
`expected_leftover`	out	expected units left over	,
`expected_profit`	out	expected profit	,
`stockout_probability`	out	probability demand exceeds Q	,

Applies when: salvage < cost < price; 0 < mean_demand <= 500; demand Poisson.

```
$ industrial-engineering solve ie_newsvendor_poisson price=20 cost=8 mean_demand=12
Q = 13
critical_ratio = 0.6
expected_sales = 11.05161977
expected_leftover = 1.948380235
expected_profit = 117.0323953
stockout_probability = 0.3184643679
  by: Newsvendor order quantity (Poisson demand)  (smallest Q with P(D <= Q) >= Cu/(Cu+Co);
E[leftover] = sum_{k<Q} (Q - k) P(D = k))
  price = 20
  cost = 8
  mean_demand = 12
  assumes: salvage < cost < price; 0 < mean_demand <= 500; demand Poisson
```

What the worked example means: This calculation determines the best single-period inventory level for Poisson demand. It compares the cost of understocking against the cost of overstocking to find the smallest integer quantity where the cumulative probability of demand meeting or exceeding that level is at least equal to the critical ratio. The resulting optimal order size is 13 units, which balances the risk of leftover stock valued at 1.94838 units against potential lost sales, yielding an expected profit of 117.032.

Newsvendor order quantity (uniform demand), `ie_newsvendor_uniform`

Single-period order quantity and expected profit when demand is uniform between two bounds.

$$Q^* = a + \frac{C_u}{C_u + C_o}(b - a)$$

symbol		meaning	SI unit
`price`	in	selling price per unit	,

symbol		meaning	SI unit
`cost`	in	purchase cost per unit	,
`low`	in	lowest possible demand	,
`high`	in	highest possible demand	,
`salvage`	in	salvage value per unsold unit (optional, default 0) (optional)	,
`Q`	out	optimal order quantity a + CR (b - a)	,
`critical_ratio`	out	critical ratio	,
`expected_sales`	out	expected units sold	,
`expected_leftover`	out	expected units left over	,
`expected_profit`	out	expected profit	,

Applies when: salvage < cost < price; high > low; demand uniform on [low, high].

```
$ industrial-engineering solve ie_newsvendor_uniform price=12 cost=7 low=100 high=300 salvage=3
Q = 211.1111111
critical_ratio = 0.5555555556
expected_sales = 180.2469136
expected_leftover = 30.86419753
expected_profit = 777.7777778
  by: Newsvendor order quantity (uniform demand)  (Q = a + CR (b - a), E[sales] = Q - (Q - a)^2 /
(2 (b - a)))
  price = 12
  cost = 7
  low = 100
  high = 300
  salvage = 3
  assumes: salvage < cost < price; high > low; demand uniform on [low, high]
```

What the worked example means: This calculation determines the best number of units to stock for a single period when demand varies uniformly between known limits. By balancing the cost of unsold items against the lost sales from running out, the model identifies an order size of 211.111 units. This decision results in an expected critical ratio of 0.555556, leading to projected sales of 180.247 units and leftover inventory of 30.8642 units.

Number of kanbans, `ie_kanban_count`

Number of kanban containers needed to cover demand during the replenishment lead time plus a safety factor.

$$N = \left\lceil \frac{d\,L\,(1+\alpha)}{C} \right\rceil$$

symbol		meaning	SI unit
`demand_rate`	in	demand per time unit (per hour, say)	,
`lead_time`	in	replenishment lead time in the same time unit (waiting + processing)	,
`container_size`	in	parts per container	,
`safety_factor`	in	safety factor (decimal; default 0) (optional)	,

symbol		meaning	SI unit
`kanbans`	out	number of kanbans (rounded up)	,
`exact`	out	d L (1 + alpha) / C before rounding	,
`max_inventory`	out	maximum inventory N C	,

Applies when: demand_rate, lead_time, container_size > 0; safety_factor >= 0.

```
$ industrial-engineering solve ie_kanban_count demand_rate=100 lead_time=2 container_size=20
safety_factor=0.1
kanbans = 11
exact = 11
max_inventory = 220
  by: Number of kanbans  (N = ceil(d L (1 + alpha) / C))
  demand_rate = 100
  lead_time = 2
  container_size = 20
  safety_factor = 0.1
  assumes: demand_rate, lead_time, container_size > 0; safety_factor >= 0
```

What the worked example means: This capability computes the number of kanban cards needed for a pull system. It uses the demand rate, replenishment lead time, container size, and a safety factor. The formula calculates the total parts needed over the lead time, adds a buffer, and divides by the container capacity. The result is rounded up to ensure sufficient coverage. For the given inputs, the unrounded value is 11, which leads to 11 physical containers. This setup ensures that inventory does not run out while waiting for replenishment.

Periodic review order-up-to level, `ie_order_up_to_level`

Order-up-to level (and order size) of a periodic-review policy that protects against demand over the review period plus the lead time.

$$S = \bar{d}(T + L) + z\sigma_d\sqrt{T + L}$$

symbol		meaning	SI unit
`mean_demand`	in	mean demand per period (per day, say)	,
`sd_demand`	in	standard deviation of demand per period	,
`review_period`	in	periods between reviews	,
`lead_time`	in	lead time in periods	,
`service_level`	in	cycle service level: probability of no stock-out per cycle (decimal)	,
`inventory_position`	in	current inventory position: on hand + on order - backorders (optional) (optional)	,
`order_up_to_level`	out	order-up-to level S = d(T+L) + z sigma sqrt(T+L)	,
`safety_stock`	out	safety stock	,
`z`	out	service factor	,
`sd_protection_demand`	out	standard deviation of demand over T + L	,
`order_quantity`	out	order size S - inventory position (when given)	,

Applies when: mean_demand, sd_demand, review_period > 0; lead_time >= 0; 0.5 < service_level < 1.

```
$ industrial-engineering solve ie_order_up_to_level mean_demand=20 sd_demand=4 review_period=14
lead_time=7 service_level=0.95 inventory_position=150
order_up_to_level = 450.150665
safety_stock = 30.15066501
z = 1.644853627
sd_protection_demand = 18.33030278
order_quantity = 300.150665
  by: Periodic review order-up-to level  (S = d (T + L) + z sigma_d sqrt(T + L); order S - IP)
  mean_demand = 20
  sd_demand = 4
  review_period = 14
  lead_time = 7
  service_level = 0.95
  inventory_position = 150
  assumes: mean_demand, sd_demand, review_period > 0; lead_time >= 0; 0.5 < service_level < 1
```

In the worked example, mean_demand = 20, sd_demand = 4, review_period = 14, lead_time = 7, service_level = 0.95, inventory_position = 150 go through the equation above and give order_up_to_level = 450.150665, safety_stock = 30.15066501, z = 1.644853627, sd_protection_demand = 18.33030278. It applies only when mean_demand, sd_demand, review_period > 0; lead_time >= 0; 0.5 < service_level < 1: outside that it says what is wrong instead of returning a number.

Reorder point (constant demand), `ie_reorder_point`

Inventory level at which to reorder: demand during the lead time plus any safety stock.

$$ROP = d\,L + SS$$

symbol		meaning	SI unit
`demand_rate`	in	demand per period (per day, say)	,
`lead_time`	in	lead time in the same periods	,
`safety_stock`	in	safety stock (optional, default 0) (optional)	,
`reorder_point`	out	reorder point d L + SS	,
`lead_time_demand`	out	demand during the lead time d L	,

Applies when: demand_rate > 0; lead_time >= 0; safety_stock >= 0.

```
$ industrial-engineering solve ie_reorder_point demand_rate=40 lead_time=5
reorder_point = 200
lead_time_demand = 200
  by: Reorder point (constant demand)  (ROP = d L + SS)
  demand_rate = 40
  lead_time = 5
  assumes: demand_rate > 0; lead_time >= 0; safety_stock >= 0
```

In the worked example, demand_rate = 40, lead_time = 5 go through the equation above and give reorder_point = 200, lead_time_demand = 200. It applies only when demand_rate > 0; lead_time >= 0; safety_stock >= 0: outside that it says what is wrong instead of returning a number.

Safety stock and reorder point (normal demand), `ie_safety_stock`

Safety stock and reorder point for a cycle service level when demand per period is normal and the lead time is fixed.

$$SS = z\sigma_d\sqrt{L}, \quad ROP = \bar{d}L + SS$$

symbol		meaning	SI unit
`mean_demand`	in	mean demand per period (per day, say)	,
`sd_demand`	in	standard deviation of demand per period	,
`lead_time`	in	lead time in periods	,
`service_level`	in	cycle service level: probability of no stock-out per cycle (decimal)	,
`safety_stock`	out	safety stock z sigma sqrt(L)	,
`reorder_point`	out	reorder point d L + SS	,
`z`	out	service factor z	,
`sd_lead_time_demand`	out	standard deviation of lead-time demand	,
`lead_time_demand`	out	mean lead-time demand	,

Applies when: mean_demand, sd_demand, lead_time > 0; 0.5 < service_level < 1; demand per period independent and normal.

```
$ industrial-engineering solve ie_safety_stock mean_demand=50 sd_demand=12 lead_time=9
service_level=0.95
safety_stock = 59.21473057
reorder_point = 509.2147306
z = 1.644853627
sd_lead_time_demand = 36
lead_time_demand = 450
  by: Safety stock and reorder point (normal demand)  (sigma_L = sigma_d sqrt(L), z = Phi^-1(CSL),
SS = z sigma_L, ROP = d L + SS)
  mean_demand = 50
  sd_demand = 12
  lead_time = 9
  service_level = 0.95
  assumes: mean_demand, sd_demand, lead_time > 0; 0.5 < service_level < 1; demand per period
independent and normal
```

What the worked example means: This calculation determines the inventory buffer needed to protect against demand variability during lead time. Given an average demand of 50 per period with a standard deviation of 12, and a fixed lead time of 9 periods, the system computes the mean lead-time demand as 450. To achieve a cycle service level of 0.95, a safety factor of 1.64485 is applied to the standard deviation of lead-time demand, which is 36. This results in a required safety stock of 59.2147. Consequently, the reorder point is set at 509.215, meaning that when inventory drops to this level, a new order should be placed to maintain the desired service probability.

Safety stock with variable demand and lead time, `ie_safety_stock_variable_lead_time`

Safety stock and reorder point when both the demand per period and the lead time vary.

$$\sigma_{DL} = \sqrt{\bar{L}\,\sigma_d^2 + \bar{d}^2\sigma_L^2}, \quad SS = z\,\sigma_{DL}$$

symbol		meaning	SI unit
`mean_demand`	in	mean demand per period (per day, say)	,
`sd_demand`	in	standard deviation of demand per period	,
`mean_lead_time`	in	mean lead time in periods	,
`sd_lead_time`	in	standard deviation of the lead time	,
`service_level`	in	cycle service level: probability of no stock-out per cycle (decimal)	,
`safety_stock`	out	safety stock z sigma_DL	,
`reorder_point`	out	reorder point d L + SS	,
`sd_lead_time_demand`	out	sigma_DL = sqrt(L sigma_d^2 + d^2 sigma_L^2)	,
`z`	out	service factor	,

Applies when: mean_demand, mean_lead_time > 0; standard deviations >= 0, not both 0; 0.5 < service_level < 1; demand and lead time independent; lead-time demand treated as normal.

```
$ industrial-engineering solve ie_safety_stock_variable_lead_time mean_demand=100 sd_demand=20
mean_lead_time=6 sd_lead_time=2 service_level=0.95
safety_stock = 338.6960916
reorder_point = 938.6960916
sd_lead_time_demand = 205.9126028
z = 1.644853627
  by: Safety stock with variable demand and lead time  (sigma_DL = sqrt(L sigma_d^2 + d^2
sigma_L^2), SS = z sigma_DL)
  mean_demand = 100
  sd_demand = 20
  mean_lead_time = 6
  sd_lead_time = 2
  service_level = 0.95
  assumes: mean_demand, mean_lead_time > 0; standard deviations >= 0, not both 0; 0.5 <
service_level < 1; demand and lead time independent; lead-time demand treated as normal
```

What the worked example means: This calculation determines the buffer inventory needed to protect against variability in both demand and lead time. Using a mean demand of 100 per period with a standard deviation of 20, and a mean lead time of 6 periods with a standard deviation of 2, this gives the combined standard deviation of lead-time demand, which is 205.913. Applying a service factor of 1.64485 for the target service level of 0.95, the resulting safety stock is 338.696. Consequently, the reorder point, where new orders should be placed, is set at 938.696.

Service level of a reorder point, `ie_service_level_from_reorder_point`

Cycle service level, expected shortage per cycle and fill rate achieved by a given reorder point with normal lead-time demand.

$$CSL = \Phi(z), \quad ESC = \sigma_L\,L(z), \quad \beta = 1 - \frac{ESC}{Q}$$

symbol		meaning	SI unit
`mean_ltd`	in	mean demand during the lead time	,
`sd_ltd`	in	standard deviation of lead-time demand	,
`reorder_point`	in	reorder point R	,
`order_quantity`	in	order quantity Q (optional; gives the fill rate) (optional)	,
`cycle_service_level`	out	P(no stock-out in a cycle) = Phi(z)	,
`z`	out	service factor (R - mu)/sigma	,
`safety_stock`	out	safety stock R - mu	,
`expected_shortage_per_cycle`	out	sigma L(z)	,
`fill_rate`	out	fill rate 1 - ESC/Q (when Q is given)	,

Applies when: sd_ltd > 0; order_quantity > 0 when given.

```
$ industrial-engineering solve ie_service_level_from_reorder_point mean_ltd=500 sd_ltd=100
reorder_point=650 order_quantity=1000
cycle_service_level = 0.9331927987
z = 1.5
safety_stock = 150
expected_shortage_per_cycle = 2.930679376
fill_rate = 0.9970693206
  by: Service level of a reorder point  (z = (R - mu)/sigma, CSL = Phi(z), ESC = sigma L(z), fill
rate = 1 - ESC/Q)
  mean_ltd = 500
  sd_ltd = 100
  reorder_point = 650
  order_quantity = 1000
  assumes: sd_ltd > 0; order_quantity > 0 when given
```

What the worked example means: This calculation determines the inventory performance metrics for a specific reorder point. Using the mean lead-time demand of 500 and standard deviation of 100, the safety stock is computed as 150. The resulting cycle service level, representing the probability of no stock-out during a replenishment cycle, is 0.933193. Additionally, the expected shortage per cycle is 2.93068, which, when combined with the order quantity of 1000, results in a fill rate of 0.997069. These values indicate how effectively the chosen reorder point meets customer demand requirements.

Learning curves (5)

Cumulative-average learning curve (Wright), `ie_learning_curve_cumulative_average`

Wright's model, where the cumulative AVERAGE time falls by the learning rate at each doubling: average, total and N-th unit time.

$$\bar{T}_N = T_1 N^b, \quad \text{total}_N = T_1 N^{b+1}$$

symbol		meaning	SI unit
`first_unit_time`	in	time (or cost) of the first unit	,
`learning_rate`	in	learning rate (decimal; 80% = 0.8)	,

symbol		meaning	SI unit
`units`	in	cumulative units N	,
`cumulative_average_time`	out	T_1 N^b	,
`total_time`	out	T_1 N^(b+1)	,
`unit_time`	out	time of unit N: total(N) - total(N - 1)	,

Applies when: first_unit_time > 0; 0 < learning_rate <= 1; units >= 1.

```
$ industrial-engineering solve ie_learning_curve_cumulative_average first_unit_time=100
learning_rate=0.8 units=4
cumulative_average_time = 64
total_time = 256
unit_time = 45.36888917
  by: Cumulative-average learning curve (Wright)  (average(N) = T_1 N^b; total = N average; unit N
= total(N) - total(N - 1))
  first_unit_time = 100
  learning_rate = 0.8
  assumes: first_unit_time > 0; 0 < learning_rate <= 1; units >= 1
```

What the worked example means: This calculation applies Wright's learning curve model to determine production efficiency metrics. Using a first unit time of 100 and a learning rate of 0.8 for 4 units, the model computes the cumulative average time per unit as 64. The total time required to produce all 4 units is calculated as 256. Additionally, the specific time required to produce only the final unit, known as the marginal unit time, is derived by subtracting the total time for the previous unit count from the current total, resulting in 45.3689. These values illustrate how labor or cost requirements decrease as cumulative production experience increases.

Fit a learning curve to data (log-log regression), `ie_learning_curve_fit`

Learning rate and first-unit time estimated by least squares on log(time) against log(unit number).

$$\ln T_n = \ln T_1 + b \ln n$$

symbol		meaning	SI unit
`units`	in	unit numbers observed	,
`times`	in	time of each observed unit	,
`learning_rate`	out	2^b	,
`exponent`	out	slope b of the log-log fit	,
`first_unit_time`	out	exp(intercept)	,
`r_squared`	out	coefficient of determination	,

Applies when: positive values; at least 2 different unit numbers.

```
$ industrial-engineering solve ie_learning_curve_fit "units=[1, 2, 4, 8, 16]" "times=[100, 82, 63,
52, 40]"
learning_rate = 0.7954830815
exponent = -0.3300968476
first_unit_time = 101.1536719
r_squared = 0.997447163
  by: Fit a learning curve to data (log-log regression)  (regress ln T on ln n: ln T = ln T_1 + b
ln n; r = 2^b)
  assumes: positive values; at least 2 different unit numbers
```

What the worked example means: This calculation fits a power law model to production data by regressing the natural logarithm of time against the natural logarithm of unit number. The resulting slope determines the learning exponent, which yields a learning rate of 0.795483. The intercept provides an estimate for the time required to produce the very first unit, calculated as 101.154. The fit quality is indicated by an r-squared value of 0.997447, showing how well the theoretical curve matches the observed times.

Learning curve: time of the n-th unit, `ie_learning_curve_unit`

Time of the n-th unit on a learning curve: each doubling of output multiplies the unit time by the learning rate (T_n = T_1 n^b, b = log2 r).

$$T_n = T_1\, n^b, \quad b = \frac{\ln r}{\ln 2}$$

symbol		meaning	SI unit
`first_unit_time`	in	time (or cost) of the first unit	,
`learning_rate`	in	learning rate (decimal; 80% = 0.8)	,
`unit`	in	unit number n	,
`unit_time`	out	T_1 n^b	,
`exponent`	out	b = log2(learning rate)	,

Applies when: first_unit_time > 0; 0 < learning_rate <= 1; unit >= 1.

```
$ industrial-engineering solve ie_learning_curve_unit first_unit_time=100 learning_rate=0.8 unit=8
unit_time = 51.2
exponent = -0.3219280949
  by: Learning curve: time of the n-th unit  (T_n = T_1 n^(log2 r))
  first_unit_time = 100
  learning_rate = 0.8
  assumes: first_unit_time > 0; 0 < learning_rate <= 1; unit >= 1
```

What the worked example means: This calculation determines the expected time or cost to produce a specific unit number using the learning curve model. It uses the time required for the very first unit, which is 100, and applies a learning rate of 0.8 to find the efficiency gain over repeated production. The resulting time for unit 8 is 51.2, reflecting how labor hours decrease as cumulative output doubles.

Learning curve: total and average time of N units, `ie_learning_curve_total`

Total and average time to make units 1 to N on a unit learning curve (sum of T_1 i^b).

$$\sum_{i=1}^{N} T_1 \, i^{b}$$

symbol		meaning	SI unit
`first_unit_time`	in	time (or cost) of the first unit	,
`learning_rate`	in	learning rate (decimal; 80% = 0.8)	,
`units`	in	number of units N	,
`total_time`	out	sum_{i=1}^{N} T_1 i^b	,
`average_time`	out	total / N	,
`last_unit_time`	out	T_1 N^b	,

Applies when: first_unit_time > 0; 0 < learning_rate <= 1; 1 <= units <= 200000.

```
$ industrial-engineering solve ie_learning_curve_total first_unit_time=100 learning_rate=0.8
units=8
total_time = 534.5913533
average_time = 66.82391917
last_unit_time = 51.2
  by: Learning curve: total and average time of N units  (total = T_1 sum_{i=1}^{N} i^b)
  first_unit_time = 100
  learning_rate = 0.8
  assumes: first_unit_time > 0; 0 < learning_rate <= 1; 1 <= units <= 200000
```

What the worked example means: This calculation determines the cumulative effort required to produce a specific quantity of items based on learning curve theory. By applying the first unit's duration and the learning rate, it sums the decreasing time required for each subsequent unit up to the target count. The result shows that as production volume increases, the average time per unit drops, reflecting improved efficiency. For the specified batch size, the total accumulated time is 534.591, while the mean time per individual item is 66.8239.

Learning rate from two unit times, `ie_learning_rate_from_two_units`

Learning rate, exponent and first-unit time of a unit learning curve through two observed (unit, time) points.

$$b = \frac{\ln(t_2/t_1)}{\ln(n_2/n_1)}, \quad r = 2^{b}$$

symbol		meaning	SI unit
`unit1`	in	first unit number	,
`time1`	in	time of that unit	,
`unit2`	in	second unit number	,
`time2`	in	time of that unit	,
`learning_rate`	out	2^b	,
`exponent`	out	b = ln(t2/t1)/ln(n2/n1)	,
`first_unit_time`	out	t1 / n1^b	,

Applies when: units and times > 0; unit numbers differ.

```
$ industrial-engineering solve ie_learning_rate_from_two_units unit1=1 time1=100 unit2=8 time2=51.2
learning_rate = 0.8
exponent = -0.3219280949
first_unit_time = 100
  by: Learning rate from two unit times  (b = ln(t2/t1) / ln(n2/n1), r = 2^b, T_1 = t1 / n1^b)
  unit1 = 1
  time1 = 100
  unit2 = 8
  time2 = 51.2
  assumes: units and times > 0; unit numbers differ
```

What the worked example means: This calculation determines the parameters of a unit learning curve using two observed data points. The exponent, calculated as -0.321928, describes how quickly performance improves as cumulative production increases. The learning rate, shown as 0.8, represents the ratio of time required for the second doubling of units compared to the first. Finally, the estimated time for the very first unit is 100. These values allow engineers to predict future labor times based on historical efficiency gains.

Line balancing (4)

Line balance efficiency, balance delay and idle time, `ie_line_efficiency`

Efficiency of a balanced line: total task time / (stations x cycle time), the balance delay and the idle time per cycle.

$$E = \frac{\sum t_i}{N\,C}, \quad \text{balance delay} = 1 - E$$

symbol		meaning	SI unit
`task_times`	in	time of each task	,
`cycle_time`	in	cycle time	,
`stations`	in	number of workstations used	,
`efficiency`	out	sum t / (N C)	,
`balance_delay`	out	1 - efficiency	,
`idle_time`	out	N C - sum t per cycle	,
`total_task_time`	out	sum of task times	,
`minimum_stations`	out	theoretical minimum stations	,

Applies when: task times > 0; each task <= cycle time; sum t <= N C.

```
$ industrial-engineering solve ie_line_efficiency "task_times=[0.5, 1.2, 0.8, 0.4, 1, 0.6]"
cycle_time=1.5 stations=4
efficiency = 0.75
balance_delay = 0.25
idle_time = 1.5
total_task_time = 4.5
minimum_stations = 3
  by: Line balance efficiency, balance delay and idle time  (E = sum t / (N C); idle = N C - sum t)
  cycle_time = 1.5
  assumes: task times > 0; each task <= cycle time; sum t <= N C
```

What the worked example means: The model calculates line balance efficiency by dividing the total task time, which is 4.5, by the product of the number of stations, 4, and the cycle time, 1.5. This yields an efficiency of 0.75. The balance delay, representing the percentage of capacity lost to imbalance, is 0.25. Consequently, the idle time per cycle is 1.5, indicating how much available station time remains unused during each production interval.

Line balancing by ranked positional weight, `ie_ranked_positional_weight`

Helgeson-Birnie method: rank tasks by positional weight (own time + times of all following tasks) and fill stations in that order.

$$RPW_j = t_j + \sum_{k \in S(j)} t_k$$

symbol		meaning	SI unit
`task_times`	in	time of each task (tasks numbered 1, 2, ...)	,
`precedence`	in	precedence rows [a, b]: task a must be done before task b	,
`cycle_time`	in	cycle time	,
`stations`	out	number of stations	,
`assignment`	out	station number of each task	,
`station_times`	out	work assigned to each station	,
`efficiency`	out	sum t / (N C)	,
`idle_time`	out	N C - sum t	,
`minimum_stations`	out	theoretical minimum stations	,
`positional_weights`	out	positional weight of each task	,

Applies when: task times > 0 and <= cycle time; precedence acyclic; ties go to the lower task number.

```
$ industrial-engineering solve ie_ranked_positional_weight "task_times=[2, 3, 4, 3, 6, 4, 2, 3]"
"precedence=[[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]]" cycle_time=10
stations = 3
assignment = [1,1,1,2,2,3,3,3]
station_times = [9.0,9.0,9.0]
efficiency = 0.9
idle_time = 3
minimum_stations = 3
positional_weights = [27.0,15.0,19.0,12.0,15.0,9.0,5.0,3.0]
  by: Line balancing by ranked positional weight  (RPW_j = t_j + sum of t over all successors of j;
assign the eligible task with the highest RPW that fits)
  cycle_time = 10
  assumes: task times > 0 and <= cycle time; precedence acyclic; ties go to the lower task number
```

What the worked example means: This calculation determines the optimal assignment of assembly tasks to workstations using the ranked positional weight method. Each task is assigned a weight equal to its own duration plus the sum of durations of all tasks that must follow it. Tasks are then placed into stations in descending order of this weight, provided they fit within the cycle time and respect precedence constraints. The result shows that 3 stations were required to complete the production line, with the total idle time across all stations being 3. This configuration yields a line efficiency of 0.9, indicating how well the available time at each station is utilized.

Line balancing by the largest candidate rule, `ie_largest_candidate_rule`

Assign tasks to stations: fill each station with the longest eligible task that still fits (all predecessors assigned), then open the next station.

$$E = \frac{\sum t_i}{N\,C}$$

symbol		meaning	SI unit
`task_times`	in	time of each task (tasks numbered 1, 2, ...)	,
`precedence`	in	precedence rows [a, b]: task a must be done before task b	,
`cycle_time`	in	cycle time	,
`stations`	out	number of stations	,
`assignment`	out	station number of each task	,
`station_times`	out	work assigned to each station	,
`efficiency`	out	sum t / (N C)	,
`idle_time`	out	N C - sum t	,
`minimum_stations`	out	theoretical minimum stations	,

Applies when: task times > 0 and <= cycle time; precedence acyclic; ties go to the lower task number.

```
$ industrial-engineering solve ie_largest_candidate_rule "task_times=[2, 3, 4, 3, 6, 4, 2, 3]"
"precedence=[[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]]" cycle_time=10
stations = 3
assignment = [1,1,1,2,2,3,3,3]
station_times = [9.0,9.0,9.0]
efficiency = 0.9
idle_time = 3
minimum_stations = 3
  by: Line balancing by the largest candidate rule  (largest candidate rule: at each station
repeatedly add the eligible task with the largest time that fits)
  cycle_time = 10
  assumes: task times > 0 and <= cycle time; precedence acyclic; ties go to the lower task number
```

What the worked example means: This calculation uses the largest candidate rule to assign tasks to workstations based on their durations and precedence constraints. The process fills each station with the longest eligible task that fits within the cycle time of 10, opening a new station when no remaining eligible task can fit. The resulting assignment required 3 stations, yielding a total idle time of 3 and a line efficiency of 0.9, which compares the sum of task times against the total available capacity.

Takt (cycle) time and minimum stations, `ie_takt_time`

Cycle time that meets demand (available production time / demand) and, with the total task time, the theoretical minimum number of stations.

$$C = \frac{\text{available time}}{\text{demand}}, \quad N_{min} = \left\lceil \frac{\sum t_i}{C} \right\rceil$$

symbol		meaning	SI unit
`available_time`	in	production time available per period (minutes, say)	,
`demand`	in	units required per period	,
`total_task_time`	in	sum of all task times (same time unit), optional (optional)	,
`cycle_time`	out	takt time = available time / demand	,
`output_rate`	out	units per time unit	,
`minimum_stations`	out	theoretical minimum stations ceil(total task time / cycle time)	,
`exact_stations`	out	total task time / cycle time	,

Applies when: inputs > 0; consistent time units.

```
$ industrial-engineering solve ie_takt_time available_time=480 demand=60 total_task_time=30
cycle_time = 8
output_rate = 0.125
minimum_stations = 4
exact_stations = 3.75
  by: Takt (cycle) time and minimum stations  (C = available time / demand; N_min = ceil(sum t /
C))
  available_time = 480
  demand = 60
  total_task_time = 30
  assumes: inputs > 0; consistent time units
```

What the worked example means: This calculation determines the takt or cycle time by dividing the available production time by the demand, resulting in a target of 8. Using this cycle time and the total task time, it computes the theoretical minimum number of workstations needed to meet demand. The exact station requirement is 3.75, which rounds up to a practical minimum of 4 stations.

Queueing (8)

Cost-optimal number of servers, `ie_queue_server_cost`

Number of servers of an M/M/c queue that minimises server cost plus the cost of customers waiting in the system per time unit.

$$TC(c) = c\,C_s + C_w\,L(c)$$

symbol		meaning	SI unit
`arrival_rate`	in	mean arrival rate lambda (per time unit)	,
`service_rate`	in	mean service rate mu of one server (per time unit)	,
`server_cost_rate`	in	cost of one server per time unit	,
`waiting_cost_rate`	in	cost of one customer in the system per time unit	,
`max_servers`	in	largest number of servers considered (default 50) (optional)	,
`servers`	out	cost-minimising number of servers	,
`total_cost`	out	c C_s + C_w L at that number	,
`L`	out	mean number in system at that number	,

Applies when: arrival_rate, service_rate > 0; costs >= 0; ties go to fewer servers.

```
$ industrial-engineering solve ie_queue_server_cost arrival_rate=20 service_rate=6
server_cost_rate=25 waiting_cost_rate=40
servers = 5
total_cost = 284.4668757
L = 3.986671893
  by: Cost-optimal number of servers  (for each stable c: L from Erlang C; total cost c C_s + C_w
L; the smallest wins)
  arrival_rate = 20
  service_rate = 6
  server_cost_rate = 25
  waiting_cost_rate = 40
  assumes: arrival_rate, service_rate > 0; costs >= 0; ties go to fewer servers
```

What the worked example means: This calculation determines the number of service channels that minimizes the combined expense of operating those servers and managing customer waiting time. By evaluating the trade-off between fixed server costs and variable waiting penalties, the model identifies 5 as the optimal configuration for the given arrival rate of 20 and service rate of 6. At this level, the expected number of customers in the system is 3.98667, leading to a minimum total hourly cost of 284.467.

Little's law, `ie_littles_law`

L = lambda W: the average number in a system equals the arrival rate times the average time spent; give any two to get the third.

$$L = \lambda W$$

symbol		meaning	SI unit
`arrival_rate`	in	average arrival (throughput) rate (optional)	,
`number_in_system`	in	average number in the system (WIP) (optional)	,
`time_in_system`	in	average time in the system (flow time) (optional)	,
`arrival_rate`	out	lambda	,
`number_in_system`	out	L = lambda W	,
`time_in_system`	out	W = L/lambda	,

Applies when: exactly two of the three given; consistent time units; steady state.

```
$ industrial-engineering solve ie_littles_law arrival_rate=12 time_in_system=0.5
arrival_rate = 12
number_in_system = 6
time_in_system = 0.5
  by: Little's law  (L = lambda W)
  arrival_rate = 12
  time_in_system = 0.5
  assumes: exactly two of the three given; consistent time units; steady state
```

What the worked example means: Little's law states that the average number of items in a stable system equals the average arrival rate multiplied by the average time each item spends in the system. Using the provided values, this gives the inventory level as 6. This result connects throughput speed with dwell time, showing how much work-in-progress accumulates when items enter at a rate of 12 and remain for 0.5. Understanding this balance helps managers align capacity with demand to prevent excessive waiting or idle resources.

M/D/1 queue (constant service time), `ie_md1_queue`

Mean queue length and waits of a single-server queue with Poisson arrivals and a constant service time.

$$L_q = \frac{\rho^2}{2(1-\rho)}$$

symbol		meaning	SI unit
`arrival_rate`	in	mean arrival rate lambda (per time unit)	,
`service_rate`	in	mean service rate mu of one server (per time unit)	,
`utilization`	out	rho = lambda/mu	,
`Lq`	out	mean number waiting rho^2/(2(1 - rho))	,
`Wq`	out	mean wait in queue Lq/lambda	,
`W`	out	mean time in system Wq + 1/mu	,
`L`	out	mean number in system Lq + rho	,

Applies when: arrival_rate < service_rate; constant service time 1/mu.

```
$ industrial-engineering solve ie_md1_queue arrival_rate=2 service_rate=3
utilization = 0.6666666667
Lq = 0.6666666667
Wq = 0.3333333333
W = 0.6666666667
L = 1.333333333
  by: M/D/1 queue (constant service time)  (Lq = rho^2 / (2 (1 - rho)), Wq = Lq/lambda)
  arrival_rate = 2
  service_rate = 3
  assumes: arrival_rate < service_rate; constant service time 1/mu
```

What the worked example means: This calculation determines the performance of a single-server queue where arrivals follow a Poisson process and service times are constant. Given an arrival rate of 2 and a service rate of 3, the server utilization is 0.666667. The resulting mean number of customers waiting in line is 0.666667, which translates to a mean waiting time before service begins of 0.333333. Including the fixed service time, the total time spent in the system averages 0.666667, with a mean number of customers in the system of 1.33333.

M/G/1 queue (Pollaczek-Khinchine), `ie_mg1_queue`

Mean queue length and waits of a single-server queue with Poisson arrivals and a general service time of given mean and standard deviation.

$$L_q = \frac{\lambda^2 \sigma^2 + \rho^2}{2(1-\rho)}$$

symbol		meaning	SI unit
`arrival_rate`	in	mean arrival rate lambda (per time unit)	,
`mean_service_time`	in	mean service time E[S]	,
`sd_service_time`	in	standard deviation of the service time	,
`utilization`	out	rho = lambda E[S]	,
`Lq`	out	Lq = lambda^2 (sigma^2 + E[S]^2)/(2(1 - rho))	,
`Wq`	out	mean wait in queue	,
`W`	out	mean time in system	,
`L`	out	mean number in system	,

Applies when: lambda E[S] < 1; sd_service_time >= 0.

```
$ industrial-engineering solve ie_mg1_queue arrival_rate=0.5 mean_service_time=1.5
sd_service_time=1
utilization = 0.75
Lq = 1.625
Wq = 3.25
W = 4.75
L = 2.375
  by: M/G/1 queue (Pollaczek-Khinchine)  (Pollaczek-Khinchine: Lq = (lambda^2 sigma^2 + rho^2)/(2(1
- rho)), Wq = Lq/lambda, W = Wq + E[S])
  arrival_rate = 0.5
  mean_service_time = 1.5
  sd_service_time = 1
  assumes: lambda E[S] < 1; sd_service_time >= 0
```

What the worked example means: This calculation determines the performance of a single-server queue with Poisson arrivals and general service times using the Pollaczek-Khinchine formula. With an arrival rate of 0.5 per unit time, a mean service time of 1.5, and a service time standard deviation of 1, the server utilization is 0.75. The resulting mean number of customers waiting in the queue is 1.625, while the total mean number in the system is 2.375. Consequently, the average time spent waiting before service begins is 3.25, and the total average time in the system is 4.75. These metrics indicate how congestion scales with both traffic intensity and service time variability.

M/M/1 queue, `ie_mm1_queue`

Utilisation, mean numbers in system and in queue, and mean times of a single-server queue with Poisson arrivals and exponential service.

$$\rho = \frac{\lambda}{\mu},\ L = \frac{\rho}{1-\rho},\ W = \frac{1}{\mu-\lambda},\ L_q = \frac{\rho^2}{1-\rho}$$

symbol		meaning	SI unit
`arrival_rate`	in	mean arrival rate lambda (per time unit)	,
`service_rate`	in	mean service rate mu of one server (per time unit)	,
`utilization`	out	rho = lambda/mu	,
`L`	out	mean number in system rho/(1 - rho)	,
`Lq`	out	mean number waiting rho^2/(1 - rho)	,
`W`	out	mean time in system 1/(mu - lambda)	,
`Wq`	out	mean wait in queue rho/(mu - lambda)	,
`p0`	out	probability the system is empty	,

Applies when: arrival_rate < service_rate; Poisson arrivals, exponential service, FCFS, infinite queue.

```
$ industrial-engineering solve ie_mm1_queue arrival_rate=2 service_rate=3
utilization = 0.6666666667
L = 2
Lq = 1.333333333
W = 1
Wq = 0.6666666667
p0 = 0.3333333333
  by: M/M/1 queue  (rho = lambda/mu, L = rho/(1 - rho), W = 1/(mu - lambda), Lq = rho L, Wq = rho
W)
  arrival_rate = 2
  service_rate = 3
  assumes: arrival_rate < service_rate; Poisson arrivals, exponential service, FCFS, infinite queue
```

What the worked example means: This calculation determines the steady-state performance of a single-server queue where arrivals follow a Poisson process and service times are exponential. Using the arrival rate 2 and service rate 3, the server utilization is 0.666667. Consequently, there are on average 2 customers in the entire system, of which 1.33333 are waiting in line. The total time a customer spends in the system averages 1, while the time spent just waiting for service averages 0.666667. The probability that the system is completely empty is 0.333333.

M/M/1/K queue (finite capacity), `ie_mm1k_queue`

Single-server queue with room for at most K customers in the system: blocking probability, effective arrival rate, mean numbers and times.

$$P_n = \frac{\rho^n}{\sum_{k=0}^{K} \rho^k}, \quad \lambda_{eff} = \lambda(1 - P_K)$$

symbol		meaning	SI unit
`arrival_rate`	in	mean arrival rate lambda (per time unit)	,
`service_rate`	in	mean service rate mu of one server (per time unit)	,
`capacity`	in	maximum customers in the system K (including the one in service)	,
`p0`	out	probability empty	,
`p_block`	out	probability an arrival is turned away P_K	,
`L`	out	mean number in system	,
`Lq`	out	mean number waiting	,
`effective_arrival_rate`	out	lambda (1 - P_K)	,
`W`	out	mean time in system L/lambda_eff	,
`Wq`	out	mean wait Lq/lambda_eff	,
`utilization`	out	1 - P0	,

Applies when: arrival_rate, service_rate > 0 (rho may exceed 1); 1 <= capacity <= 1000.

```
$ industrial-engineering solve ie_mm1k_queue arrival_rate=4 service_rate=5 capacity=3
p0 = 0.3387533875
p_block = 0.1734417344
L = 1.224932249
Lq = 0.5636856369
effective_arrival_rate = 3.306233062
W = 0.3704918033
Wq = 0.1704918033
utilization = 0.6612466125
  by: M/M/1/K queue (finite capacity)  (P_n = rho^n / sum_{k=0}^{K} rho^k; lambda_eff = lambda (1 -
P_K); Little's law)
  arrival_rate = 4
  service_rate = 5
  assumes: arrival_rate, service_rate > 0 (rho may exceed 1); 1 <= capacity <= 1000
```

What the worked example means: This calculation determines the steady-state behavior of a single-server queue with a maximum capacity of 3 customers. Given an arrival rate of 4 and a service rate of 5, it computes the probability that the system is empty, which is 0.338753, and the probability that an arriving customer is blocked, which is 0.173442. The mean number of customers in the system is found to be 1.22493, while the effective arrival rate of customers who actually enter the system is 3.30623. Consequently, the average time a customer spends in the system is 0.370492.

M/M/c queue (Erlang C), `ie_mmc_queue`

Probability of waiting (Erlang C), mean queue length and mean waits of a multi-server queue with Poisson arrivals and exponential service.

$$C(c,a) = \frac{\frac{a^c}{c!(1-\rho)}}{\sum_{k=0}^{c-1}\frac{a^k}{k!} + \frac{a^c}{c!(1-\rho)}}, \quad L_q = C\frac{\rho}{1-\rho}$$

symbol		meaning	SI unit
`arrival_rate`	in	mean arrival rate lambda (per time unit)	,
`service_rate`	in	mean service rate mu of one server (per time unit)	,
`servers`	in	number of servers c	,
`utilization`	out	rho = lambda/(c mu)	,
`p_wait`	out	Erlang C probability of waiting	,
`Lq`	out	mean number waiting	,
`Wq`	out	mean wait in queue	,
`W`	out	mean time in system	,
`L`	out	mean number in system	,
`p0`	out	probability the system is empty	,

Applies when: arrival_rate < servers x service_rate; 1 <= servers <= 500.

```
$ industrial-engineering solve ie_mmc_queue arrival_rate=10 service_rate=4 servers=3
utilization = 0.8333333333
p_wait = 0.702247191
Lq = 3.511235955
Wq = 0.3511235955
W = 0.6011235955
L = 6.011235955
p0 = 0.04494382022
  by: M/M/c queue (Erlang C)  (a = lambda/mu, P0 = [sum_{k<c} a^k/k! + a^c/(c!(1-rho))]^-1, C = a^c
P0/(c!(1-rho)), Lq = C rho/(1-rho))
  arrival_rate = 10
  service_rate = 4
  assumes: arrival_rate < servers x service_rate; 1 <= servers <= 500
```

What the worked example means: This calculation determines the steady-state performance of a multi-server queueing system using Erlang C formulas. Given an arrival rate of 10 and a service rate of 4 across 3 servers, the system operates at a utilization of 0.833333. The resulting probability that an arriving customer must wait for service is 0.702247. Consequently, the mean number of customers waiting in line is 3.51124, and the average time spent waiting before service begins is 0.351124. The total mean time in the system, including service, is 0.601124, with a mean population of 6.01124.

Servers needed for a service level (Erlang C), `ie_erlang_c_staffing`

Fewest servers of an M/M/c queue such that the probability of waiting no longer than a target time reaches a target service level (call-centre staffing).

$$P(W_q \le t) = 1 - C(c, a)\, e^{-(c\mu - \lambda)t}$$

symbol		meaning	SI unit
`arrival_rate`	in	mean arrival rate lambda (per time unit)	,
`service_rate`	in	mean service rate mu of one server (per time unit)	,
`target_time`	in	target answer time in the same time unit	,
`target_level`	in	required fraction answered within the target time (decimal)	,
`servers`	out	fewest servers meeting the target	,
`service_level`	out	P(wait <= t) achieved	,
`p_wait`	out	Erlang C probability of waiting	,
`utilization`	out	lambda/(c mu)	,
`mean_wait`	out	mean wait in queue	,

Applies when: arrival_rate, service_rate > 0; target_time >= 0; 0 < target_level < 1.

```
$ industrial-engineering solve ie_erlang_c_staffing arrival_rate=100 service_rate=12
target_time=0.0333 target_level=0.8
servers = 11
service_level = 0.8967748179
p_wait = 0.29961935
utilization = 0.7575757576
mean_wait = 0.009363104689
  by: Servers needed for a service level (Erlang C)  (smallest c > lambda/mu with 1 - C(c, a) exp(-
(c mu - lambda) t) >= target)
  arrival_rate = 100
  service_rate = 12
  target_time = 0.0333
  target_level = 0.8
  assumes: arrival_rate, service_rate > 0; target_time >= 0; 0 < target_level < 1
```

What the worked example means: This calculation determines the minimum number of servers required in a queueing system to ensure that a specified fraction of customers waits no longer than a given time. Using an Erlang C model, it finds the smallest integer count of servers where the probability of waiting within the target duration meets or exceeds the desired service level. For the provided inputs, the result is 11 servers, achieving a service level of 0.896775, with each server having a utilization of 0.757576.

Reliability (13)

Cold standby redundancy, `ie_standby_redundancy`

Reliability and MTTF of one operating unit backed by identical cold standby units with perfect switching.

$$R(t) = e^{-\lambda t} \sum_{k=0}^{n-1} \frac{(\lambda t)^k}{k!}$$

symbol		meaning	SI unit
`failure_rate`	in	failure rate lambda of each unit	,
`time`	in	mission time	,
`units`	in	total units including the operating one (default 2) (optional)	,
`reliability`	out	exp(-lambda t) sum_{k<n} (lambda t)^k / k!	,
`mttf`	out	n/lambda	,

Applies when: failure_rate > 0; time >= 0; 1 <= units <= 200; standby units do not fail, switching is perfect.

```
$ industrial-engineering solve ie_standby_redundancy failure_rate=0.001 time=1000
reliability = 0.7357588823
mttf = 2000
  by: Cold standby redundancy  (R(t) = e^(-lambda t) sum_{k=0}^{n-1} (lambda t)^k/k!)
  failure_rate = 0.001
  time = 1000
  assumes: failure_rate > 0; time >= 0; 1 <= units <= 200; standby units do not fail, switching is
perfect
```

What the worked example means: This calculation determines the probability that the system remains operational over the specified mission duration. It uses an exponential model for unit failures, assuming identical components with perfect switching from cold standby states. The result, 0.735759, indicates the fraction of such systems expected to survive without failure up to time 1000. Additionally, the mean time to failure is computed as 2000, representing the average lifespan of the redundant configuration.

Confidence bounds on MTBF (chi-square), `ie_mtbf_confidence`

Point estimate and chi-square confidence bounds of the MTBF from a life test with constant failure rate.

$$MTBF_L = \frac{2T}{\chi^2_{C,\,2r+2}}, \quad MTBF_U = \frac{2T}{\chi^2_{\alpha/2,\,2r}}$$

symbol		meaning	SI unit
`total_time`	in	total accumulated test time (all units)	,
`failures`	in	number of failures observed	,
`confidence`	in	confidence level (decimal)	,
`test`	in	time (test stopped at a set time) or failure (stopped at the r-th failure); default time (optional)	,
`mtbf`	out	point estimate T/r	,
`lower_one_sided`	out	one-sided lower bound 2T/chi2(C; d)	,
`lower_two_sided`	out	two-sided lower bound	,
`upper_two_sided`	out	two-sided upper bound 2T/chi2(a/2; 2r)	,

Applies when: total_time > 0; failures >= 1; 0 < confidence < 1; d = 2r + 2 for a time-terminated test, 2r for a failure-terminated test.

```
$ industrial-engineering solve ie_mtbf_confidence total_time=10000 failures=4 confidence=0.9
test=time
mtbf = 2500
lower_one_sided = 1251.00243
lower_two_sided = 1092.476016
upper_two_sided = 7318.938268
  by: Confidence bounds on MTBF (chi-square)  (MTBF_L = 2T / chi2_{C, d}; two-sided: 2T/chi2_{1-
a/2, d} and 2T/chi2_{a/2, 2r})
  total_time = 10000
  confidence = 0.9
  test = time
  assumes: total_time > 0; failures >= 1; 0 < confidence < 1; d = 2r + 2 for a time-terminated
test, 2r for a failure-terminated test
```

What the worked example means: This calculation determines the mean time between failures from accumulated test data. The central estimate is 2500, derived from the total operating time and observed failures. To account for statistical uncertainty, a one-sided lower bound of 1251 is provided based on the specified confidence level. Two-sided bounds are also calculated as 1092.48 and 7318.94, offering a range within which the true reliability metric likely falls.

Exponential (constant failure rate) reliability, `ie_exponential_reliability`

Probability of surviving a mission time with a constant failure rate: R(t) = exp(-lambda t), MTBF = 1/lambda.

$$R(t) = e^{-\lambda t}, \quad MTBF = \frac{1}{\lambda}$$

symbol		meaning	SI unit
`failure_rate`	in	constant failure rate lambda (failures per time unit)	,
`time`	in	mission time in the same time unit	,
`reliability`	out	R(t) = exp(-lambda t)	,
`unreliability`	out	F(t) = 1 - R(t)	,
`mtbf`	out	mean time between failures 1/lambda	,
`hazard_rate`	out	lambda	,

Applies when: failure_rate > 0; time >= 0.

```
$ industrial-engineering solve ie_exponential_reliability failure_rate=0.0002 time=1000
reliability = 0.8187307531
unreliability = 0.1812692469
mtbf = 5000
hazard_rate = 0.0002
  by: Exponential (constant failure rate) reliability  (R = exp(-lambda t))
  failure_rate = 0.0002
  time = 1000
  assumes: failure_rate > 0; time >= 0
```

What the worked example means: This calculation determines the exponential reliability, which represents the probability that a component or system will function without failure for a specified duration. By applying the formula where reliability equals e raised to the negative product of the failure rate and time, this gives the result to be 0.818731. This value is derived from an input failure rate of 0.0002 per unit time over a mission period of 1000. The corresponding mean time between failures is calculated as 5000, while the unreliability, representing the chance of failure within that window, is 0.181269.

Failure rate and MTBF of a series system, `ie_system_failure_rate`

Failure rate, MTBF and mission reliability of a series system of components with constant failure rates.

$$\lambda_s = \sum_i \lambda_i, \quad MTBF_s = \frac{1}{\lambda_s}$$

symbol		meaning	SI unit
`failure_rates`	in	constant failure rate of each component (per hour, say)	,
`time`	in	mission time (optional) (optional)	,
`system_failure_rate`	out	sum of lambda_i	,
`mtbf`	out	1 / sum lambda_i	,
`reliability`	out	exp(-sum lambda_i t) (when a time is given)	,

Applies when: failure rates > 0; time >= 0; series system, constant failure rates.

```
$ industrial-engineering solve ie_system_failure_rate "failure_rates=[0.0001, 0.0002, 5e-05]"
time=1000
system_failure_rate = 0.00035
mtbf = 2857.142857
reliability = 0.7046880897
  by: Failure rate and MTBF of a series system  (lambda_s = sum lambda_i, MTBF = 1/lambda_s, R(t) =
exp(-lambda_s t))
  time = 1000
  assumes: failure rates > 0; time >= 0; series system, constant failure rates
```

What the worked example means: This calculation determines the overall performance of a series system by summing the individual constant failure rates of all components to find the system failure rate, which is 0.00035. The reciprocal of this total rate provides the mean time between failures, calculated as 2857.14. If a specific mission duration is provided, such as 1000, the probability that the system survives without failure is computed using an exponential decay model, resulting in a reliability value of 0.704688. These metrics help assess how component-level vulnerabilities aggregate into system-level risk.

Inherent availability from MTBF and MTTR, `ie_availability`

Long-run fraction of time a repairable system is up, A = MTBF / (MTBF + MTTR), and the downtime it implies.

$$A = \frac{MTBF}{MTBF + MTTR}$$

symbol		meaning	SI unit
`mtbf`	in	mean time between failures (hours)	,
`mttr`	in	mean time to repair (hours)	,
`hours_per_year`	in	operating hours in a year (default 8760) (optional)	,
`availability`	out	MTBF/(MTBF + MTTR)	,
`unavailability`	out	MTTR/(MTBF + MTTR)	,
`downtime_per_year`	out	expected downtime hours per year	,
`failure_rate`	out	1/MTBF	,
`repair_rate`	out	1/MTTR	,

Applies when: mtbf, mttr > 0.

```
$ industrial-engineering solve ie_availability mtbf=500 mttr=20
availability = 0.9615384615
unavailability = 0.03846153846
downtime_per_year = 336.9230769
failure_rate = 0.002
repair_rate = 0.05
  by: Inherent availability from MTBF and MTTR  (A = MTBF/(MTBF + MTTR))
  mtbf = 500
  mttr = 20
  assumes: mtbf, mttr > 0
```

What the worked example means: Inherent availability is 0.961538, implying 336.923 hours of downtime per year.

k-out-of-n system reliability, `ie_k_out_of_n_reliability`

Reliability of a system of n identical independent components that works when at least k of them work.

$$R = \sum_{j=k}^{n} \binom{n}{j} r^j (1-r)^{n-j}$$

symbol		meaning	SI unit
`component_reliability`	in	reliability of each component	,
`k`	in	components needed	,
`n`	in	components installed	,
`reliability`	out	P(at least k of n work)	,
`unreliability`	out	1 - reliability	,

Applies when: 0 <= r <= 1; 1 <= k <= n <= 1000; identical independent components.

```
$ industrial-engineering solve ie_k_out_of_n_reliability component_reliability=0.9 k=2 n=3
reliability = 0.972
unreliability = 0.028
  by: k-out-of-n system reliability  (R = sum_{j=k}^{n} C(n, j) r^j (1 - r)^(n - j))
  component_reliability = 0.9
  assumes: 0 <= r <= 1; 1 <= k <= n <= 1000; identical independent components
```

What the worked example means: This calculation determines the reliability of a k-out-of-n system, which operates successfully if at least k out of n identical independent components are working. Given a component reliability of 0.9, the resulting system reliability is 0.972. Consequently, the unreliability, or the probability of system failure, is 0.028. This metric helps assess how redundancy affects overall system performance.

MTTF of identical components in parallel, `ie_parallel_mttf`

Mean time to failure of n identical exponential components in active parallel: (1/lambda) sum 1/i.

$$MTTF = \frac{1}{\lambda} \sum_{i=1}^{n} \frac{1}{i}$$

symbol		meaning	SI unit
`failure_rate`	in	failure rate lambda of each component	,
`n`	in	number of components in parallel	,
`mttf`	out	system MTTF (1/lambda) sum_{i=1}^{n} 1/i	,
`single_mttf`	out	1/lambda	,
`improvement_factor`	out	sum 1/i	,

Applies when: failure_rate > 0; 1 <= n <= 1000; active redundancy, constant failure rate.

```
$ industrial-engineering solve ie_parallel_mttf failure_rate=0.001 n=2
mttf = 1500
single_mttf = 1000
improvement_factor = 1.5
  by: MTTF of identical components in parallel  (MTTF = integral of 1 - (1 - e^(-lambda t))^n dt =
(1/lambda) sum 1/i)
  failure_rate = 0.001
  assumes: failure_rate > 0; 1 <= n <= 1000; active redundancy, constant failure rate
```

What the worked example means: This calculation determines the system reliability metric known as Mean Time To Failure, or MTTF, for a configuration where multiple identical components operate simultaneously. By taking the individual failure rate and the count of parallel units, the model computes the expected operational lifespan before total system failure. The result shows that with 2 components, the system achieves an improvement factor of 1.5 over a single unit, yielding a final MTTF of 1500. This value represents the average time the redundant system will function correctly under constant stress conditions.

Parallel components needed for a target reliability, `ie_redundancy_needed`

Smallest number of identical components in parallel that reaches a target system reliability.

$$m = \left\lceil \frac{\ln(1 - R_{target})}{\ln(1 - r)} \right\rceil$$

symbol		meaning	SI unit
`target_reliability`	in	required system reliability	,
`component_reliability`	in	reliability of one component	,
`components`	out	smallest m with 1 - (1 - r)^m >= target	,
`achieved_reliability`	out	reliability with m components	,

Applies when: 0 < target < 1; 0 < r < 1; active parallel redundancy.

```
$ industrial-engineering solve ie_redundancy_needed target_reliability=0.999
component_reliability=0.9
components = 3
achieved_reliability = 0.999
  by: Parallel components needed for a target reliability  (smallest m with 1 - (1 - r)^m >=
R_target (m = ceil(ln(1 - R)/ln(1 - r))))
  target_reliability = 0.999
  component_reliability = 0.9
  assumes: 0 < target < 1; 0 < r < 1; active parallel redundancy
```

What the worked example means: This algorithm computes the minimum number of parallel components needed to meet a specific system reliability goal. Given a single component reliability of 0.9, it finds the smallest integer count, reported as 3, such that the combined probability of failure is low enough to satisfy the requirement of 0.999. The resulting achieved reliability for this configuration is 0.999, ensuring the system meets or exceeds the specified safety or performance threshold.

Reliability of components in parallel, `ie_parallel_reliability`

Reliability of a redundant system that works if at least one component works (independent components).

$$R_p = 1 - \prod_{i=1}^{n}(1 - R_i)$$

symbol		meaning	SI unit
`reliabilities`	in	reliability of each component (decimals)	,
`reliability`	out	1 - product (1 - R_i)	,
`unreliability`	out	product (1 - R_i)	,

Applies when: each reliability in [0, 1]; independent failures; active redundancy.

```
$ industrial-engineering solve ie_parallel_reliability "reliabilities=[0.9, 0.8]"
reliability = 0.98
unreliability = 0.02
  by: Reliability of components in parallel  (R = 1 - product (1 - R_i))
  assumes: each reliability in [0, 1]; independent failures; active redundancy
```

What the worked example means: This calculation determines the overall reliability of a redundant system where the system functions if at least one component works. It uses the formula that subtracts the product of all individual unreliabilities from one. The resulting system reliability is 0.98, which represents the probability that the system will perform its intended function under stated conditions for a specified period of time, assuming independent failures and active redundancy.

Reliability of components in series, `ie_series_reliability`

Reliability of a system that works only if every component works (independent components).

$$R_s = \prod_{i=1}^{n} R_i$$

symbol		meaning	SI unit
`reliabilities`	in	reliability of each component (decimals)	,
`reliability`	out	system reliability product of R_i	,
`unreliability`	out	1 - system reliability	,

Applies when: each reliability in [0, 1]; independent failures.

```
$ industrial-engineering solve ie_series_reliability "reliabilities=[0.95, 0.98, 0.99]"
reliability = 0.92169
unreliability = 0.07831
  by: Reliability of components in series  (R = product R_i)
  assumes: each reliability in [0, 1]; independent failures
```

What the worked example means: This calculation determines the overall reliability of a series system by multiplying the reliability values of all independent components. The result, 0.92169, represents the

probability that the entire system functions correctly without failure. Conversely, the unreliability, calculated as 0.07831, indicates the likelihood that at least one component fails, causing the whole system to stop working.

Series system of parallel stages, `ie_series_parallel_reliability`

Reliability of stages in series where each stage has redundant components in parallel (each row of the matrix is one stage).

$$R = \prod_{i} \left[1 - \prod_{j} (1 - r_{ij}) \right]$$

symbol		meaning	SI unit
`stage_reliabilities`	in	matrix: one row per stage, the reliabilities of its parallel components	,
`reliability`	out	product over stages of 1 - product (1 - r)	,
`unreliability`	out	1 - reliability	,

Applies when: every value in [0, 1]; independent components; rows of equal length.

```
$ industrial-engineering solve ie_series_parallel_reliability "stage_reliabilities=[[0.9, 0.9],
[0.8, 0.8], [0.95, 0.95]]"
reliability = 0.948024
unreliability = 0.051976
  by: Series system of parallel stages  (R = prod_i [1 - prod_j (1 - r_ij)])
  assumes: every value in [0, 1]; independent components; rows of equal length
```

What the worked example means: This calculation determines the total reliability for a system composed of multiple stages connected in series, where each stage contains several components arranged in parallel. By treating each row of the input matrix as a distinct stage and applying the product rule to the inner parallel terms, this gives the likelihood that at least one component functions in every stage. The resulting value, 0.948024, represents the cumulative probability that the entire chain operates without failure, while 0.051976 quantifies the residual risk of system downtime.

Weibull fit by median-rank regression, `ie_weibull_median_rank_regression`

Weibull shape and scale estimated from complete failure times by regression on Bernard's median ranks (Weibull probability plot).

$$\ln \ln \frac{1}{1 - F_i} = \beta \ln t_{(i)} - \beta \ln \eta, \quad F_i = \frac{i - 0.3}{n + 0.4}$$

symbol		meaning	SI unit
`failure_times`	in	failure times (complete data, any order)	,
`shape`	out	shape beta (slope of ln ln(1/(1-F)) on ln t)	,
`scale`	out	scale eta	,
`r_squared`	out	coefficient of determination of the plot	,

Applies when: at least 3 positive failure times, not all equal; complete (uncensored) data.

```
$ industrial-engineering solve ie_weibull_median_rank_regression "failure_times=[16, 34, 53, 75,
93, 120]"
shape = 1.426967111
scale = 76.34541546
r_squared = 0.9911811558
  by: Weibull fit by median-rank regression  (F_i = (i - 0.3)/(n + 0.4); regress y = ln(-ln(1 -
F_i)) on x = ln t_(i); beta = slope, eta = exp(-intercept/beta))
  assumes: at least 3 positive failure times, not all equal; complete (uncensored) data
```

What the worked example means: This calculation estimates the parameters of a two-parameter Weibull distribution from a set of complete failure times. It uses median rank regression, where each ordered time is assigned a probability based on its rank, and then fits a straight line to the transformed data. The resulting shape parameter is 1.42697 and the scale parameter is 76.3454. A higher shape value indicates that failures are becoming more frequent over time, suggesting wear-out behavior, while a lower value suggests early-life failures or constant hazard rates.

Weibull reliability, hazard and MTTF, `ie_weibull_reliability`

Reliability, hazard rate, mean life and B10 life of a Weibull life distribution (shape beta, scale eta, optional location gamma).

$$R(t) = \exp\left[-\left(\frac{t-\gamma}{\eta}\right)^{\beta}\right], \quad MTTF = \gamma + \eta\Gamma(1+1/\beta)$$

symbol		meaning	SI unit
`shape`	in	shape parameter beta	,
`scale`	in	scale parameter eta (characteristic life)	,
`time`	in	time t	,
`location`	in	location gamma, failure-free time (default 0) (optional)	,
`reliability`	out	R(t) = exp(-((t - gamma)/eta)^beta)	,
`unreliability`	out	1 - R(t)	,
`hazard_rate`	out	h(t) = beta/eta ((t - gamma)/eta)^(beta - 1)	,
`mttf`	out	gamma + eta Gamma(1 + 1/beta)	,
`b10_life`	out	time by which 10 % have failed	,

Applies when: shape, scale > 0; location >= 0; time >= location.

```
$ industrial-engineering solve ie_weibull_reliability shape=2 scale=1000 time=500
reliability = 0.7788007831
unreliability = 0.2211992169
hazard_rate = 0.001
mttf = 886.2269255
b10_life = 324.592846
  by: Weibull reliability, hazard and MTTF  (R = exp(-((t-g)/eta)^b); MTTF = g + eta Gamma(1 +
1/b); B10 = g + eta (-ln 0.9)^(1/b))
  shape = 2
  scale = 1000
  time = 500
  assumes: shape, scale > 0; location >= 0; time >= location
```

What the worked example means: The Weibull reliability at time 500 is 0.778801, with a mean life of 886.227.

Scheduling (7)

Earliest due date (EDD) sequence, `ie_edd_sequence`

Order jobs on one machine by earliest due date (minimises the maximum lateness) and report tardiness measures.

$$d_{[1]} \le d_{[2]} \le \dots, \quad L_{max} = \max_{k}(C_{[k]} - d_{[k]})$$

symbol		meaning	SI unit
`processing_times`	in	processing time of each job (jobs numbered 1, 2, ... in this order)	,
`due_dates`	in	due date of each job (same order)	,
`sequence`	out	job numbers in processing order	,
`mean_flow_time`	out	average completion (flow) time	,
`makespan`	out	completion time of the last job	,
`average_jobs_in_system`	out	sum of completion times / makespan	,
`utilization`	out	sum of processing times / sum of completion times	,
`total_tardiness`	out	sum of tardiness	,
`mean_tardiness`	out	average tardiness (job lateness)	,
`max_tardiness`	out	largest tardiness	,
`max_lateness`	out	largest lateness C - d	,
`tardy_jobs`	out	number of late jobs	,

Applies when: processing times > 0; due dates >= 0; ties go to the lower job number.

```
$ industrial-engineering solve ie_edd_sequence "processing_times=[4, 7, 2, 5]" "due_dates=[10, 15,
5, 12]"
sequence = [3,1,4,2]
mean_flow_time = 9.25
makespan = 18
average_jobs_in_system = 2.055555556
utilization = 0.4864864865
total_tardiness = 3
mean_tardiness = 0.75
max_tardiness = 3
max_lateness = 3
tardy_jobs = 1
  by: Earliest due date (EDD) sequence  (sort by due date; lateness C - d; tardiness max(0, C - d))
  assumes: processing times > 0; due dates >= 0; ties go to the lower job number
```

What the worked example means: This method sorts all jobs by their due dates from earliest to latest. The processing times for the ordered sequence determine the completion time for each job. The result shows the optimal order is [3, 1, 4, 2], which yields an average flow time of 9.25 and a total tardiness of 3. This approach ensures that the largest deviation between completion and due date, known as maximum lateness, is kept at 3.

Johnson's rule (two machines), `ie_johnson_two_machine`

Sequence jobs through two machines in the same order so that the makespan is minimal (Johnson's rule).

$$\min(a_{1i}, a_{2j}) \leq \min(a_{1j}, a_{2i}) \Rightarrow i \text{ before } j$$

symbol		meaning	SI unit
`machine1_times`	in	time of each job on machine 1 (first)	,
`machine2_times`	in	time of each job on machine 2 (second)	,
`sequence`	out	job order	,
`makespan`	out	time all jobs finish on machine 2	,
`idle_time_machine2`	out	idle time of machine 2 (makespan - its work)	,

Applies when: times >= 0; jobs with a1 <= a2 go first by increasing a1, the others last by decreasing a2; ties by the lower job number.

```
$ industrial-engineering solve ie_johnson_two_machine "machine1_times=[5, 2, 1, 7, 6]"
"machine2_times=[2, 6, 2, 5, 3]"
sequence = [3,2,4,5,1]
makespan = 23
idle_time_machine2 = 5
  by: Johnson's rule (two machines)  (Johnson: front set (a1 <= a2) by a1 ascending, back set by a2
descending)
  assumes: times >= 0; jobs with a1 <= a2 go first by increasing a1, the others last by decreasing
a2; ties by the lower job number
```

What the worked example means: This calculation applies Johnson's rule to determine the best order for processing jobs on two machines. By comparing the processing time on the first machine against the second, jobs are split into two groups: those where the first machine time is less than or equal to the second are

placed at the beginning sorted by increasing first machine time, while the rest are placed at the end sorted by decreasing second machine time. The resulting sequence [3, 2, 4, 5, 1] minimizes the total completion time, known as the makespan, which equals 23. This method ensures that idle time on the second machine is minimized whenever possible.

Johnson's rule extended to three machines, `ie_johnson_three_machine`

Optimal job order through three machines when machine 2 is dominated (Johnson's extension with pseudo times a1 + a2 and a2 + a3).

$$a'_i = a_{1i} + a_{2i}, \quad a''_i = a_{2i} + a_{3i}$$

symbol		meaning	SI unit
`machine1_times`	in	time of each job on machine 1	,
`machine2_times`	in	time of each job on machine 2	,
`machine3_times`	in	time of each job on machine 3	,
`sequence`	out	job order	,
`makespan`	out	time all jobs finish on machine 3	,

Applies when: min(machine 1) >= max(machine 2) or min(machine 3) >= max(machine 2) (else refused); ties by the lower job number.

```
$ industrial-engineering solve ie_johnson_three_machine "machine1_times=[8, 10, 6, 7, 11]"
"machine2_times=[5, 6, 2, 3, 4]" "machine3_times=[4, 9, 8, 6, 5]"
sequence = [3,2,1,4,5]
makespan = 51
  by: Johnson's rule extended to three machines  (Johnson's rule on pseudo machines (a1 + a2, a2 +
a3); makespan by the three-machine recursion)
  assumes: min(machine 1) >= max(machine 2) or min(machine 3) >= max(machine 2) (else refused);
ties by the lower job number
```

What the worked example means: This calculation determines the best order for jobs to pass through three machines using an extension of Johnson's rule. It first checks if the middle machine is dominated by either the first or third machine. If so, it creates two pseudo-machines with times equal to the sum of adjacent real machines. The algorithm then applies standard Johnson's logic to these pseudo-times to find the sequence that minimizes the total completion time, which is reported as 51.

Minimise the number of late jobs (Moore-Hodgson), `ie_moore_hodgson`

Sequence on one machine that minimises the number of late jobs: on-time jobs in EDD order, then the late ones (Moore-Hodgson algorithm).

$$\min \sum_j U_j, \quad U_j = [C_j > d_j]$$

symbol		meaning	SI unit
`processing_times`	in	processing time of each job (jobs numbered 1, 2, ... in this order)	,

symbol		meaning	SI unit
`due_dates`	in	due date of each job (same order)	,
`number_late`	out	least possible number of late jobs	,
`late_jobs`	out	the jobs scheduled late	,
`on_time_jobs`	out	the on-time jobs in processing (EDD) order	,
`sequence`	out	full sequence	,

Applies when: processing times > 0; due dates >= 0; when a job is late, the longest scheduled job is removed (ties: the lower job number).

```
$ industrial-engineering solve ie_moore_hodgson "processing_times=[4, 3, 2, 5, 6]" "due_dates=[5,
6, 8, 10, 12]"
number_late = 2
late_jobs = [1,5]
on_time_jobs = [2,3,4]
sequence = [2,3,4,1,5]
  by: Minimise the number of late jobs (Moore-Hodgson)  (add jobs in EDD order; whenever the
current job finishes late, drop the longest job scheduled so far)
  assumes: processing times > 0; due dates >= 0; when a job is late, the longest scheduled job is
removed (ties: the lower job number)
```

What the worked example means: This calculation determines the optimal job sequence to minimize lateness using the Moore-Hodgson algorithm. It processes jobs in earliest due date order, removing the longest job whenever a deadline is missed. The result identifies that 2 jobs will be late, specifically [1, 5], while the remaining jobs [2, 3, 4] are scheduled first in an efficient order.

Permutation flow shop makespan, `ie_flow_shop_makespan`

Makespan and completion times of jobs processed in a given order through machines in series (each row a job in processing order, each column a machine).

$$C_{i,j} = \max(C_{i-1,j},\, C_{i,j-1}) + t_{i,j}$$

symbol		meaning	SI unit
`times`	in	matrix: row = job (in processing order), column = machine (in route order)	,
`makespan`	out	completion time of the last job on the last machine	,
`completion_times`	out	completion time of each job on the last machine	,
`idle_time_last_machine`	out	idle time of the last machine	,

Applies when: times >= 0; the same job order on every machine, no passing.

```
$ industrial-engineering solve ie_flow_shop_makespan "times=[[5, 3, 6], [2, 7, 4], [4, 2, 5], [6,
4, 3]]"
makespan = 27
completion_times = [14.0,19.0,24.0,27.0]
idle_time_last_machine = 9
  by: Permutation flow shop makespan  (C(i, j) = max(C(i-1, j), C(i, j-1)) + t(i, j))
  assumes: times >= 0; the same job order on every machine, no passing
```

What the worked example means: This calculation determines the total time required to process all jobs through a series of machines, assuming each job follows the same sequence. The result, known as the makespan, is 27. This value represents the moment the final job exits the last machine. Additionally, the system provides individual completion times for each job on the final stage, denoted as [14, 19, 24, 27], and calculates how long the last machine remained idle before finishing its work, recorded as 9. These metrics help evaluate the efficiency of the chosen job order.

Sequencing by a dispatching rule (FCFS, SPT, LPT, EDD, CR), `ie_dispatching_rule`

Sequence jobs at one work centre by a priority rule and compare average completion time, utilisation, average number of jobs and tardiness.

$$\bar{F} = \frac{\sum C_j}{n}, \quad \text{utilization} = \frac{\sum p_j}{\sum C_j}, \quad \bar{N} = \frac{\sum C_j}{C_{max}}$$

symbol		meaning	SI unit
`processing_times`	in	processing time of each job (jobs numbered 1, 2, ... in this order)	,
`due_dates`	in	due date of each job (same order)	,
`rule`	in	fcfs (given order), spt, lpt (longest first), edd or cr (critical ratio due date / processing time at time 0, smallest first)	,
`sequence`	out	job numbers in processing order	,
`mean_flow_time`	out	average completion (flow) time	,
`makespan`	out	completion time of the last job	,
`average_jobs_in_system`	out	sum of completion times / makespan	,
`utilization`	out	sum of processing times / sum of completion times	,
`total_tardiness`	out	sum of tardiness	,
`mean_tardiness`	out	average tardiness (job lateness)	,
`max_tardiness`	out	largest tardiness	,
`max_lateness`	out	largest lateness C - d	,
`tardy_jobs`	out	number of late jobs	,

Applies when: processing times > 0; due dates >= 0; ties go to the lower job number; jobs available at time 0.

```
$ industrial-engineering solve ie_dispatching_rule "processing_times=[6, 2, 8, 3, 9]" "due_dates=
[8, 6, 18, 15, 23]" rule=fcfs
sequence = [1,2,3,4,5]
mean_flow_time = 15.4
makespan = 28
average_jobs_in_system = 2.75
utilization = 0.3636363636
total_tardiness = 11
mean_tardiness = 2.2
max_tardiness = 5
max_lateness = 5
tardy_jobs = 3
  by: Sequencing by a dispatching rule (FCFS, SPT, LPT, EDD, CR)  (order the jobs by the rule;
completion times are running sums; tardiness max(0, C - d))
  rule = fcfs
  assumes: processing times > 0; due dates >= 0; ties go to the lower job number; jobs available at
time 0
```

What the worked example means: This calculation orders the jobs based on the selected dispatching rule to determine the processing sequence. From this sequence, it computes key performance metrics including the average completion time, utilization, and tardiness measures. The results show that the mean flow time is 15.4, the makespan is 28, and the total tardiness is 11. These values help evaluate how effectively the chosen rule balances speed against meeting due dates.

Shortest processing time (SPT) sequence, `ie_spt_sequence`

Order jobs on one machine by shortest processing time first (minimises mean flow time) and report the usual measures.

$$p_{[1]} \le p_{[2]} \le \dots, \quad \bar{F} = \frac{1}{n}\sum_{k} C_{[k]}$$

symbol		meaning	SI unit
`processing_times`	in	processing time of each job (jobs numbered 1, 2, ... in this order)	,
`due_dates`	in	due date of each job (same order) (optional)	,
`sequence`	out	job numbers in processing order	,
`mean_flow_time`	out	average completion (flow) time	,
`makespan`	out	completion time of the last job	,
`average_jobs_in_system`	out	sum of completion times / makespan	,
`utilization`	out	sum of processing times / sum of completion times	,
`total_tardiness`	out	sum of tardiness	,
`mean_tardiness`	out	average tardiness (job lateness)	,
`max_tardiness`	out	largest tardiness	,
`max_lateness`	out	largest lateness C - d	,
`tardy_jobs`	out	number of late jobs	,

Applies when: processing times > 0; all jobs available at time 0; ties go to the lower job number; tardiness measures only when due dates are given.

```
$ industrial-engineering solve ie_spt_sequence "processing_times=[6, 2, 8, 3, 9]"
sequence = [2,4,1,3,5]
mean_flow_time = 13
makespan = 28
average_jobs_in_system = 2.321428571
utilization = 0.4307692308
  by: Shortest processing time (SPT) sequence  (sort by processing time; completion C_k = sum of
the first k times; flow = mean C)
  assumes: processing times > 0; all jobs available at time 0; ties go to the lower job number;
tardiness measures only when due dates are given
```

What the worked example means: The optimal sequence is [2, 4, 1, 3, 5] with a mean flow time of 13.

Statistical process control (18)

Average run length of a Shewhart X-bar chart, `ie_arl_shewhart`

Probability that a Shewhart chart signals on a sample after the mean shifts by k sigma, and the average run length (ARL) until it does.

$$\beta = \Phi(L - k\sqrt{n}) - \Phi(-L - k\sqrt{n}), \quad ARL_1 = \frac{1}{1-\beta}$$

symbol		meaning	SI unit
`shift`	in	mean shift in units of the process standard deviation (k)	,
`subgroup_size`	in	observations per subgroup (default 1) (optional)	,
`limit_sigma`	in	control limits at +/- L sigma of the mean (default 3) (optional)	,
`arl`	out	average run length after the shift 1/(1 - beta)	,
`beta`	out	probability of no signal per sample	,
`detection_probability`	out	probability of a signal per sample	,
`arl_in_control`	out	in-control ARL 1/alpha	,

Applies when: shift >= 0; 1 <= subgroup_size <= 1000; 0 < limit_sigma <= 6.

```
$ industrial-engineering solve ie_arl_shewhart shift=1 subgroup_size=5
arl = 4.495312227
beta = 0.7775460414
detection_probability = 0.2224539586
arl_in_control = 370.3983473
  by: Average run length of a Shewhart X-bar chart  (beta = Phi(L - k sqrt(n)) - Phi(-L - k
sqrt(n)); ARL = 1/(1 - beta))
  shift = 1
  assumes: shift >= 0; 1 <= subgroup_size <= 1000; 0 < limit_sigma <= 6
```

What the worked example means: The average run length is 4.49531, meaning the chart signals after about that many samples when the mean shifts by 1.

c chart control limits, `ie_c_chart_limits`

3-sigma limits of the count-of-defects (c) chart: c-bar +/- 3 sqrt(c-bar).

$$\bar{c} \pm 3\sqrt{\bar{c}}$$

symbol		meaning	SI unit
`c_bar`	in	average number of defects per unit (inspection unit of constant size)	,
`ucl`	out	upper control limit	,
`lcl`	out	lower control limit (not below 0)	,
`center`	out	centre line	,
`sigma_c`	out	sqrt(c-bar)	,

Applies when: c_bar > 0; defects Poisson, constant inspection unit.

```
$ industrial-engineering solve ie_c_chart_limits c_bar=9
ucl = 18
lcl = 0
center = 9
sigma_c = 3
  by: c chart control limits  (limits c +/- 3 sqrt(c))
  c_bar = 9
  assumes: c_bar > 0; defects Poisson, constant inspection unit
```

What the worked example means: This calculation determines the three-sigma control limits for a c chart, which monitors the count of defects per unit. Using the average defect rate of 9, the standard deviation is estimated as 3. The resulting upper limit of 18 and lower limit of 0 define the expected range of variation when the process is stable.

DPMO and sigma level, `ie_dpmo_sigma_level`

Defects per unit, per opportunity and per million opportunities, and the sigma level (with the conventional 1.5 sigma shift).

$$DPMO = \frac{10^6\, D}{U\, O}, \quad \mathrm{sigma} = \Phi^{-1}(1 - DPO) + 1.5$$

symbol		meaning	SI unit
`defects`	in	number of defects found	,
`units`	in	number of units inspected	,
`opportunities`	in	defect opportunities per unit	,
`dpmo`	out	defects per million opportunities	,
`dpu`	out	defects per unit	,
`dpo`	out	defects per opportunity	,
`yield`	out	1 - DPO	,
`sigma_level`	out	short-term sigma level Phi^-1(1 - DPO) + 1.5	,

Applies when: defects >= 0; units, opportunities > 0; 0 < DPO < 1.

```
$ industrial-engineering solve ie_dpmo_sigma_level defects=21 units=500 opportunities=8
dpmo = 5250
dpu = 0.042
dpo = 0.00525
yield = 0.99475
sigma_level = 4.05891305
  by: DPMO and sigma level  (DPO = defects/(units opportunities), DPMO = 1e6 DPO, sigma = Phi^-1(1
- DPO) + 1.5)
  defects = 21
  units = 500
  opportunities = 8
  assumes: defects >= 0; units, opportunities > 0; 0 < DPO < 1
```

What the worked example means: This calculation determines the process capability by analyzing defect rates. Using 21 observed defects, 500 inspected units, and 8 potential error points per unit, this gives the Defects Per Million Opportunities (DPMO). The result is 5250 DPMO, which corresponds to a long-term sigma level of 4.05891, incorporating the standard 1.5 sigma shift to account for process drift over time.

EWMA chart control limits, `ie_ewma_limits`

Control limits of an exponentially weighted moving average chart at sample i and in the steady state.

$$\mu_0 \pm L\sigma\sqrt{\frac{\lambda}{2-\lambda}[1-(1-\lambda)^{2i}]}$$

symbol		meaning	SI unit
`target`	in	in-control mean mu0	,
`sigma`	in	standard deviation of the plotted observations	,
`smoothing`	in	smoothing constant lambda (0 < lambda <= 1)	,
`limit_sigma`	in	limit width L (often 2.7 to 3)	,
`sample_number`	in	sample number i	,
`ucl`	out	UCL at sample i	,
`lcl`	out	LCL at sample i	,
`ucl_steady`	out	steady-state UCL	,
`lcl_steady`	out	steady-state LCL	,
`sd_ewma`	out	standard deviation of the EWMA at sample i	,

Applies when: sigma > 0; 0 < lambda <= 1; L > 0; sample_number >= 1.

Run `industrial-engineering info ie_ewma_limits` for its inputs, then `industrial-engineering solve ie_ewma_limits name=value ...`.

What the worked example means: This calculation determines the upper and lower control limits for an Exponentially Weighted Moving Average chart at a specific sample point. Using the in-control mean 10, process standard deviation 1, smoothing constant 0.1, and limit width 2.7, the system computes the limits to

be 9.73 and 10.27. The standard deviation of the EWMA statistic at this stage is calculated as 0.1. These boundaries help practitioners identify when the process mean has shifted significantly from its target value.

Fraction nonconforming (normal process), `ie_fraction_nonconforming`

Fraction (and parts per million) of output outside the specification limits for a normal process.

$$p = \Phi\left(\frac{LSL - \mu}{\sigma}\right) + 1 - \Phi\left(\frac{USL - \mu}{\sigma}\right)$$

symbol		meaning	SI unit
`mean`	in	process mean	,
`sd`	in	process standard deviation	,
`lsl`	in	lower specification limit (optional) (optional)	,
`usl`	in	upper specification limit (optional) (optional)	,
`fraction_out`	out	fraction outside the limits	,
`below_lsl`	out	fraction below LSL	,
`above_usl`	out	fraction above USL	,
`ppm`	out	parts per million out of specification	,
`yield`	out	fraction within specification	,

Applies when: sd > 0; at least one limit; usl > lsl; process normal.

```
$ industrial-engineering solve ie_fraction_nonconforming mean=10.1 sd=0.12 lsl=9.7 usl=10.5
fraction_out = 0.0008581206664
below_lsl = 0.0004290603332
above_usl = 0.0004290603332
ppm = 858.1206664
yield = 0.9991418793
  by: Fraction nonconforming (normal process)  (P(X < LSL) + P(X > USL) for X ~ N(mu, sigma^2))
  mean = 10.1
  sd = 0.12
  lsl = 9.7
  usl = 10.5
  assumes: sd > 0; at least one limit; usl > lsl; process normal
```

What the worked example means: This calculation determines the proportion of production that falls outside the acceptable quality range defined by the lower and upper specification limits. Assuming the process follows a normal distribution with mean 10.1 and standard deviation 0.12, this gives the probability of measurements being below 9.7 or above 10.5. The total fraction nonconforming is 0.000858121, meaning that for every one million units produced, approximately 858.121 are expected to be defective. Conversely, the yield, or the fraction within specifications, is 0.999142.

np chart control limits, `ie_np_chart_limits`

3-sigma limits of the number-defective (np) chart: n p +/- 3 sqrt(n p (1 - p)).

$$n\bar{p} \pm 3\sqrt{n\bar{p}(1-\bar{p})}$$

symbol		meaning	SI unit
`p_bar`	in	average fraction defective (decimal)	,
`sample_size`	in	items per sample	,
`ucl`	out	upper control limit	,
`lcl`	out	lower control limit (not below 0)	,
`center`	out	centre line n p	,
`sigma_np`	out	standard deviation of the number defective	,

Applies when: 0 < p_bar < 1; constant sample size.

```
$ industrial-engineering solve ie_np_chart_limits p_bar=0.04 sample_size=100
ucl = 9.878775383
lcl = 0
center = 4
sigma_np = 1.959591794
  by: np chart control limits  (limits n p +/- 3 sqrt(n p (1 - p)))
  p_bar = 0.04
  assumes: 0 < p_bar < 1; constant sample size
```

What the worked example means: This calculation determines the three-sigma control limits for an np chart, which tracks the number of defective items in a constant sample size. Using the average fraction defective of 0.04 and a sample size of 100, the process mean is estimated as 4. The standard deviation of the count is 1.95959, resulting in an upper control limit of 9.87878 and a lower control limit of 0. These boundaries help identify when the production process exhibits special cause variation rather than common chance fluctuations.

p chart control limits, `ie_p_chart_limits`

3-sigma limits of the fraction-defective (p) chart: p-bar +/- 3 sqrt(p-bar (1 - p-bar) / n).

$$UCL, LCL = \bar{p} \pm 3\sqrt{\frac{\bar{p}(1-\bar{p})}{n}}$$

symbol		meaning	SI unit
`p_bar`	in	average fraction defective (decimal)	,
`sample_size`	in	items per sample	,
`ucl`	out	upper control limit	,
`lcl`	out	lower control limit (not below 0)	,
`center`	out	centre line	,
`sigma_p`	out	standard deviation of the fraction defective	,

Applies when: 0 < p_bar < 1; sample_size > 0 (whole number of items).

```
$ industrial-engineering solve ie_p_chart_limits p_bar=0.04 sample_size=100
ucl = 0.09878775383
lcl = 0
center = 0.04
sigma_p = 0.01959591794
  by: p chart control limits  (sigma_p = sqrt(p (1 - p)/n); limits p +/- 3 sigma_p, clipped to [0,
1])
  p_bar = 0.04
  assumes: 0 < p_bar < 1; sample_size > 0 (whole number of items)
```

What the worked example means: This calculation determines the three-sigma control limits for a p-chart, which monitors the proportion of defective items in a process. Using an average fraction defective of 0.04 and a sample size of 100, the standard deviation of the proportion is found to be 0.0195959. The resulting upper control limit of 0.0987878 and lower control limit of 0 define the boundaries within which sample proportions are expected to fall if the process remains stable. These limits help identify special causes of variation that require investigation.

p chart from sample counts, `ie_p_chart_from_samples`

Centre line and 3-sigma limits of a p chart computed from the defectives found in each sample, and the samples that fall outside them.

$$\bar{p} = \frac{\sum d_i}{m\,n}, \quad \bar{p} \pm 3\sqrt{\bar{p}(1-\bar{p})/n}$$

symbol		meaning	SI unit
`defectives`	in	number of defective items in each sample	,
`sample_size`	in	items in every sample	,
`p_bar`	out	average fraction defective	,
`ucl`	out	upper control limit	,
`lcl`	out	lower control limit	,
`out_of_control`	out	sample numbers outside the limits	,
`count_out`	out	how many samples are outside	,

Applies when: counts are whole numbers <= sample size; 0 < p-bar < 1; constant sample size.

```
$ industrial-engineering solve ie_p_chart_from_samples "defectives=[3, 5, 2, 4, 12, 3, 4, 2, 5, 3]"
sample_size=100
p_bar = 0.043
ucl = 0.1038571278
lcl = 0
out_of_control = [5]
count_out = 1
  by: p chart from sample counts  (p-bar = total defectives / total inspected; limits p-bar +/- 3
sqrt(p-bar (1 - p-bar)/n); points beyond the limits flagged)
  assumes: counts are whole numbers <= sample size; 0 < p-bar < 1; constant sample size
```

What the worked example means: This calculation determines the center line and three-sigma control limits for a p chart based on the total number of defectives found across all samples. The average fraction defective,

denoted as 0.043, serves as the central reference point. Upper and lower control limits are set at 0.103857 and 0 respectively to define the expected range of variation when the process is stable. Any sample where the proportion of defectives falls outside these boundaries is flagged as out of control, indicating a special cause of variation that requires investigation.

Process capability Cp and Cpk, `ie_process_capability`

Capability indices of a process with known mean and standard deviation against two specification limits.

$$C_p = \frac{USL - LSL}{6\sigma}, \quad C_{pk} = \min\left(\frac{USL - \mu}{3\sigma}, \frac{\mu - LSL}{3\sigma}\right)$$

symbol		meaning	SI unit
`usl`	in	upper specification limit	,
`lsl`	in	lower specification limit	,
`mean`	in	process mean	,
`sd`	in	process standard deviation	,
`cpk`	out	Cpk = min(Cpu, Cpl)	,
`cp`	out	Cp = (USL - LSL)/(6 sigma)	,
`cpu`	out	(USL - mean)/(3 sigma)	,
`cpl`	out	(mean - LSL)/(3 sigma)	,

Applies when: usl > lsl; sd > 0; process normal and in control.

```
$ industrial-engineering solve ie_process_capability usl=10.5 lsl=9.5 mean=10.1 sd=0.12
cpk = 1.111111111
cp = 1.388888889
cpu = 1.111111111
cpl = 1.666666667
  by: Process capability Cp and Cpk  (Cp = (USL - LSL)/6 sigma, Cpk = min(USL - mu, mu - LSL)/3
sigma)
  usl = 10.5
  lsl = 9.5
  mean = 10.1
  sd = 0.12
  assumes: usl > lsl; sd > 0; process normal and in control
```

What the worked example means: This calculation determines the potential and actual capability of a manufacturing or service process. The index Cp, calculated as 1.38889, measures the width of the specification range relative to six times the standard deviation, showing how well the process could perform if perfectly centered. The index Cpk, calculated as 1.11111, accounts for the process mean's location by taking the minimum of the upper and lower capability ratios. A higher value indicates less variation and better alignment with specifications, while values below one suggest the process is not capable of meeting requirements consistently.

Process performance Pp and Ppk from data, `ie_process_performance`

Performance indices Pp and Ppk from measured data, using the overall sample standard deviation (n - 1).

$$P_p = \frac{USL - LSL}{6s}, \quad P_{pk} = \min\left(\frac{USL - \bar{x}}{3s}, \frac{\bar{x} - LSL}{3s}\right)$$

symbol		meaning	SI unit
`data`	in	measurements	,
`usl`	in	upper specification limit	,
`lsl`	in	lower specification limit	,
`ppk`	out	Ppk	,
`pp`	out	Pp	,
`mean`	out	sample mean	,
`sd`	out	overall sample standard deviation	,

Applies when: at least 2 measurements that vary; usl > lsl.

```
$ industrial-engineering solve ie_process_performance "data=[10.1, 9.9, 10.2, 10, 10.3, 9.8, 10.1,
10]" usl=10.6 lsl=9.4
ppk = 1.143284202
pp = 1.247219129
mean = 10.05
sd = 0.1603567451
  by: Process performance Pp and Ppk from data  (s = sqrt(sum (x - mean)^2/(n - 1)); Pp = (USL -
LSL)/6s, Ppk = min(USL - mean, mean - LSL)/3s)
  usl = 10.6
  lsl = 9.4
  assumes: at least 2 measurements that vary; usl > lsl
```

What the worked example means: This calculation determines the potential capability (Pp) and actual performance (Ppk) of a process using measured data. The sample mean is computed as 10.05, while the overall standard deviation, reflecting total variation including shifts, is 0.160357. Pp, which assesses spread relative to specification width regardless of centering, equals 1.24722. Ppk, which accounts for how centered the process is between the lower limit 9.4 and upper limit 10.6, equals 1.14328. These values indicate whether the current process output consistently falls within acceptable quality boundaries.

Process standard deviation from the average range, `ie_sigma_from_range`

Estimate of the process standard deviation sigma = R-bar / d2 and the natural tolerance 6 sigma.

$$\hat{\sigma} = \frac{\bar{R}}{d_2}$$

symbol		meaning	SI unit
`mean_range`	in	average subgroup range (R-bar)	,
`subgroup_size`	in	observations per subgroup (2 to 10)	,
`sigma`	out	estimated process standard deviation R-bar/d2	,
`d2`	out	d2 constant	,
`natural_tolerance`	out	natural tolerance 6 sigma	,

Applies when: mean_range > 0; subgroup size 2 to 10.

```
$ industrial-engineering solve ie_sigma_from_range mean_range=2 subgroup_size=5
sigma = 0.8598452279
d2 = 2.326
natural_tolerance = 5.159071367
  by: Process standard deviation from the average range  (sigma = R-bar / d2)
  mean_range = 2
  assumes: mean_range > 0; subgroup size 2 to 10
```

What the worked example means: This calculation estimates the process standard deviation by dividing the average subgroup range, given as 2, by the appropriate d2 constant. The resulting standard deviation is 0.859845, which leads to a natural tolerance width of 5.15907. This method provides a robust measure of variability for processes where data is collected in small subgroups.

Rolled throughput yield, `ie_rolled_throughput_yield`

Probability that a unit passes every process step defect-free (product of step yields), its geometric mean per step and the equivalent total defects per unit -ln(RTY).

$$RTY = \prod_i Y_i, \quad Y_{norm} = RTY^{1/k}$$

symbol		meaning	SI unit
`yields`	in	first-pass yield of each step (decimals)	,
`rty`	out	rolled throughput yield	,
`normalized_yield`	out	normalized yield RTY^(1/k)	,
`total_dpu`	out	equivalent total defects per unit -ln RTY	,

Applies when: each yield in (0, 1].

```
$ industrial-engineering solve ie_rolled_throughput_yield "yields=[0.95, 0.98, 0.9, 0.99]"
rty = 0.829521
normalized_yield = 0.9543481726
total_dpu = 0.1869068532
  by: Rolled throughput yield  (RTY = product of yields; normalized = RTY^(1/k); DPU = -ln RTY)
  assumes: each yield in (0, 1]
```

What the worked example means: This calculation determines the overall probability that a single unit passes through every process step without any defects, known as the rolled throughput yield. It is computed by multiplying the first-pass yields of all individual steps together. The result shows that the total success rate across the entire chain is 0.829521. To understand performance on an average per-step basis, the geometric mean of this total yield is calculated, resulting in a normalized yield of 0.954348. Additionally, the method converts this yield into an equivalent defect density using a logarithmic transformation, indicating that there are approximately 0.186907 defects per unit if the errors were distributed uniformly.

Single sampling plan: OC, AOQ and ATI, `ie_acceptance_sampling`

Probability of accepting a lot under a single sampling plan (n, c), the average outgoing quality with rectifying inspection and the average total inspection.

$$P_a = \sum_{k=0}^{c} \binom{n}{k} p^k (1-p)^{n-k}, \quad AOQ = \frac{P_a \, p \, (N-n)}{N}$$

symbol		meaning	SI unit
`sample_size`	in	items sampled n	,
`acceptance_number`	in	acceptance number c	,
`fraction_defective`	in	lot fraction defective p (decimal)	,
`lot_size`	in	lot size N	,
`probability_accept`	out	P(accept) = P(X <= c), X ~ Binomial(n, p)	,
`aoq`	out	average outgoing quality Pa p (N - n)/N	,
`ati`	out	average total inspection n + (1 - Pa)(N - n)	,
`probability_reject`	out	1 - Pa	,

Applies when: 0 <= c < n <= N; 0 < p < 1; binomial sampling; n <= 2000.

```
$ industrial-engineering solve ie_acceptance_sampling sample_size=50 acceptance_number=1
fraction_defective=0.02 lot_size=2000
probability_accept = 0.7357713945
aoq = 0.01434754219
ati = 565.2457808
probability_reject = 0.2642286055
  by: Single sampling plan: OC, AOQ and ATI  (Pa = sum_{k<=c} C(n,k) p^k (1-p)^(n-k); AOQ = Pa p
(N-n)/N; ATI = n + (1 - Pa)(N - n))
  fraction_defective = 0.02
  assumes: 0 <= c < n <= N; 0 < p < 1; binomial sampling; n <= 2000
```

In the worked example, sample_size = 50, acceptance_number = 1, fraction_defective = 0.02, lot_size = 2000 go through the equation above and give probability_accept = 0.7357713945, aoq = 0.01434754219, ati = 565.2457808, probability_reject = 0.2642286055. It applies only when 0 <= c < n <= N; 0 < p < 1; binomial sampling; n <= 2000: outside that it says what is wrong instead of returning a number.

Tabular CUSUM chart, `ie_tabular_cusum`

Upper and lower cumulative sums C+ and C- of a tabular CUSUM chart and the first sample that signals (C+ or C- above h).

$$C_i^+ = \max(0,\, x_i - (\mu_0 + K) + C_{i-1}^+), \quad C_i^- = \max(0,\, (\mu_0 - K) - x_i + C_{i-1}^-)$$

symbol		meaning	SI unit
`data`	in	observations in time order	,
`target`	in	target mean mu0	,
`k`	in	reference value K (often half the shift to detect)	,
`h`	in	decision interval H	,

symbol		meaning	SI unit
`first_signal`	out	first sample number with C+ or C- above H (0 = none)	,
`c_plus`	out	C+ after each sample	,
`c_minus`	out	C- after each sample	,
`final_c_plus`	out	last C+	,
`final_c_minus`	out	last C-	,

Applies when: k >= 0; h > 0; C+ and C- start at 0.

Run `industrial-engineering info ie_tabular_cusum` for its inputs, then `industrial-engineering solve ie_tabular_cusum name=value ...`.

What the worked example means: This algorithm computes the cumulative sums for detecting small shifts in the process mean. Using observations [9.45, 7.99, 9.29, 11.66, 12.16, 10.18, 8.04, 11.46, 9.2, 10.34, 11.8, 12.2, 11.9, 12.5], it tracks the upper deviation from the target 10 plus reference 0.5 and the lower deviation from the target minus reference 0.5. The first signal occurs when either sum exceeds the decision interval 5, which happened at sample 14. The final accumulated values are 6.4 for the upper side and 0 for the lower side, indicating the magnitude of any persistent drift.

Taguchi capability index Cpm, `ie_cpm_taguchi`

Capability index that penalises distance from the target: Cpm = (USL - LSL) / (6 sqrt(sigma^2 + (mu - T)^2)).

$$C_{pm} = \frac{USL - LSL}{6\sqrt{\sigma^2 + (\mu - T)^2}}$$

symbol		meaning	SI unit
`usl`	in	upper specification limit	,
`lsl`	in	lower specification limit	,
`mean`	in	process mean	,
`sd`	in	process standard deviation	,
`target`	in	target value T	,
`cpm`	out	Cpm	,
`cp`	out	Cp	,
`tau`	out	root mean square deviation from target	,

Applies when: usl > lsl; lsl < target < usl; sd > 0.

Run `industrial-engineering info ie_cpm_taguchi` for its inputs, then `industrial-engineering solve ie_cpm_taguchi name=value ...`.

What the worked example means: This calculation determines the Taguchi capability index, which assesses process performance by penalizing deviations from the target value. The root mean square deviation tau combines the process standard deviation and the squared distance between the process mean and the target. The final index divides the specification width by six times this combined deviation. With an upper limit of 10.6 and a lower limit of 9.4, a mean of 10.1, and a standard deviation of 0.1, the resulting index of 1.41421 indicates how well the process centers on the target while maintaining variation within limits.

u chart control limits, `ie_u_chart_limits`

3-sigma limits of the defects-per-unit (u) chart for samples of n units: u-bar +/- 3 sqrt(u-bar / n).

$$\bar{u} \pm 3\sqrt{\bar{u}/n}$$

symbol		meaning	SI unit
`u_bar`	in	average defects per unit	,
`units`	in	inspection units in the sample	,
`ucl`	out	upper control limit	,
`lcl`	out	lower control limit (not below 0)	,
`center`	out	centre line	,
`sigma_u`	out	sqrt(u-bar / n)	,

Applies when: u_bar > 0; units > 0.

```
$ industrial-engineering solve ie_u_chart_limits u_bar=1.5 units=10
ucl = 2.661895004
lcl = 0.3381049961
center = 1.5
sigma_u = 0.3872983346
  by: u chart control limits  (limits u +/- 3 sqrt(u/n))
  u_bar = 1.5
  units = 10
  assumes: u_bar > 0; units > 0
```

What the worked example means: This calculation establishes the three-sigma control limits for a u-chart, which tracks defects per unit. Using an average defect rate of 1.5 across a sample size of 10 units, the system computes a standard deviation estimate of 0.387298. The resulting center line sits at 1.5, with the upper control limit set to 2.6619 and the lower control limit floored at zero if necessary, yielding 0.338105. These boundaries help identify whether the process variation is due to common causes or if special cause intervention is required.

X-bar and R chart control limits, `ie_xbar_r_limits`

3-sigma control limits of the X-bar and R charts from the grand mean, the average range and the subgroup size.

$$UCL_{\bar{x}} = \bar{\bar{x}} + A_2\bar{R}, \quad UCL_R = D_4\bar{R}, \quad LCL_R = D_3\bar{R}$$

symbol		meaning	SI unit
`grand_mean`	in	grand mean of the subgroup means (x-double-bar)	,
`mean_range`	in	average subgroup range (R-bar)	,
`subgroup_size`	in	observations per subgroup (2 to 10)	,
`xbar_ucl`	out	X-bar chart UCL	,
`xbar_lcl`	out	X-bar chart LCL	,

symbol		meaning	SI unit
`center`	out	X-bar chart centre line	,
`r_ucl`	out	R chart UCL D4 R-bar	,
`r_lcl`	out	R chart LCL D3 R-bar	,
`A2`	out	A2	,
`D3`	out	D3	,
`D4`	out	D4	,

Applies when: mean_range > 0; subgroup size 2 to 10; standard 3-decimal constants.

```
$ industrial-engineering solve ie_xbar_r_limits grand_mean=50 mean_range=2 subgroup_size=5
xbar_ucl = 51.154
xbar_lcl = 48.846
center = 50
r_ucl = 4.228
r_lcl = 0
A2 = 0.577
D3 = 0
D4 = 2.114
  by: X-bar and R chart control limits  (UCL/LCL = x-double-bar +/- A2 R-bar; R chart D3 R-bar, D4
R-bar)
  grand_mean = 50
  mean_range = 2
  assumes: mean_range > 0; subgroup size 2 to 10; standard 3-decimal constants
```

What the worked example means: This calculation determines the three-sigma control limits for an X-bar and R chart based on process data. The upper limit for the mean chart is found by adding the product of the A2 constant and the average range to the grand mean, resulting in 51.154. The lower limit is derived by subtracting this same product from the grand mean, yielding 48.846. The center line remains fixed at the grand mean value of 50. Simultaneously, the range chart limits are established using the D4 and D3 constants multiplied by the average range, producing an upper limit of 4.228 and a lower limit of 0. These values define the boundaries within which process variation is considered stable.

X-bar and S chart control limits, `ie_xbar_s_limits`

3-sigma control limits of the X-bar and S charts from the grand mean, the average subgroup standard deviation and the subgroup size.

$$UCL_{\bar{x}} = \bar{\bar{x}} + A_3\bar{s}, \quad UCL_s = B_4\bar{s}, \quad LCL_s = B_3\bar{s}$$

symbol		meaning	SI unit
`grand_mean`	in	grand mean of the subgroup means	,
`mean_sd`	in	average subgroup standard deviation (s-bar)	,
`subgroup_size`	in	observations per subgroup (2 to 10)	,
`xbar_ucl`	out	X-bar chart UCL	,
`xbar_lcl`	out	X-bar chart LCL	,
`center`	out	centre line	,

symbol		meaning	SI unit
`s_ucl`	out	S chart UCL B4 s-bar	,
`s_lcl`	out	S chart LCL B3 s-bar	,
`A3`	out	A3	,
`B3`	out	B3	,
`B4`	out	B4	,

Applies when: mean_sd > 0; subgroup size 2 to 10; standard 3-decimal constants.

```
$ industrial-engineering solve ie_xbar_s_limits grand_mean=50 mean_sd=0.9 subgroup_size=5
xbar_ucl = 51.2843
xbar_lcl = 48.7157
center = 50
s_ucl = 1.8801
s_lcl = 0
A3 = 1.427
B3 = 0
B4 = 2.089
  by: X-bar and S chart control limits  (x-double-bar +/- A3 s-bar; S chart B3 s-bar, B4 s-bar)
  grand_mean = 50
  mean_sd = 0.9
  assumes: mean_sd > 0; subgroup size 2 to 10; standard 3-decimal constants
```

What the worked example means: This calculation determines the three-sigma control limits for an X-bar and S chart using the grand mean of 50 and the average subgroup standard deviation of 0.9. The center line is set at 50, with upper and lower limits derived from constants A3, B3, and B4 based on a subgroup size of 5. These boundaries help identify whether process variation remains within expected statistical limits.

Work measurement (5)

Capacity utilisation and efficiency, `ie_capacity_utilization`

Utilisation (actual output / design capacity) and efficiency (actual output / effective capacity).

$$U = \frac{\text{actual}}{\text{design}}, \quad E = \frac{\text{actual}}{\text{effective}}$$

symbol		meaning	SI unit
`design_capacity`	in	design capacity	,
`effective_capacity`	in	effective capacity	,
`actual_output`	in	actual output	,
`utilization`	out	actual / design	,
`efficiency`	out	actual / effective	,
`effective_share`	out	effective / design	,

Applies when: capacities > 0; effective <= design; same units and period.

```
$ industrial-engineering solve ie_capacity_utilization design_capacity=1200 effective_capacity=1000
actual_output=800
utilization = 0.6666666667
efficiency = 0.8
effective_share = 0.8333333333
  by: Capacity utilisation and efficiency  (utilisation = actual/design, efficiency =
actual/effective)
  design_capacity = 1200
  effective_capacity = 1000
  actual_output = 800
  assumes: capacities > 0; effective <= design; same units and period
```

What the worked example means: This calculation determines how much of the available production potential is actually used. Utilization compares the actual output to the theoretical maximum design capacity, while efficiency measures performance against the realistic effective capacity that accounts for constraints like maintenance or staffing. The result shows utilization at 0.666667 and efficiency at 0.8. Additionally, the ratio of effective to design capacity is 0.833333, indicating how much of the theoretical limit is practically attainable.

Number of cycles to time in a time study, `ie_time_study_sample_size`

Observations needed so the mean cycle time is within +/- h (a fraction of the mean) at a confidence level: n = (z s / (h x-bar))^2.

$$n = \left(\frac{z\,s}{h\,\bar{x}}\right)^2$$

symbol		meaning	SI unit
`mean`	in	mean of the pilot observations	,
`sd`	in	standard deviation of the pilot observations	,
`accuracy`	in	desired accuracy as a fraction of the mean (decimal; 5% = 0.05)	,
`confidence`	in	confidence level (decimal)	,
`sample_size`	out	observations needed (rounded up)	,
`exact`	out	(z s / (h x-bar))^2	,
`z`	out	two-sided z	,

Applies when: mean, sd > 0; 0 < accuracy < 1; 0 < confidence < 1; normal approximation.

```
$ industrial-engineering solve ie_time_study_sample_size mean=3 sd=0.5 accuracy=0.05
confidence=0.95
sample_size = 43
exact = 42.68287579
z = 1.959963985
  by: Number of cycles to time in a time study  (n = (z s / (h x-bar))^2, z for a two-sided
confidence)
  mean = 3
  sd = 0.5
  accuracy = 0.05
  confidence = 0.95
  assumes: mean, sd > 0; 0 < accuracy < 1; 0 < confidence < 1; normal approximation
```

What the worked example means: This calculation determines how many times a task must be observed in a time study to ensure the average cycle time is estimated with sufficient precision. Using a pilot sample mean of 3 and standard deviation of 0.5, along with a desired accuracy fraction of 0.05 and a confidence level of 0.95, the method computes the necessary sample size. The resulting requirement is 43 observations, derived from an exact calculated value of 42.6829. This ensures that the true mean cycle time falls within the specified margin of error around the observed average with the stated probability.

Overall equipment effectiveness (OEE), `ie_oee`

OEE = availability x performance x quality from planned time, downtime, ideal cycle time and counts.

$$OEE = A \times P \times Q = \frac{\text{good} \times t_{ideal}}{T_{planned}}$$

symbol		meaning	SI unit
`planned_time`	in	planned production time	,
`downtime`	in	stop time within the planned time	,
`ideal_cycle_time`	in	ideal time per unit (same time unit)	,
`total_count`	in	units produced	,
`good_count`	in	good units	,
`oee`	out	availability x performance x quality	,
`availability`	out	run time / planned time	,
`performance`	out	ideal cycle time x total count / run time	,
`quality`	out	good / total	,
`run_time`	out	planned time - downtime	,

Applies when: downtime < planned time; good <= total; performance <= 1.

```
$ industrial-engineering solve ie_oee planned_time=420 downtime=47 ideal_cycle_time=1
total_count=300 good_count=285
oee = 0.6785714286
availability = 0.8880952381
performance = 0.8042895442
quality = 0.95
run_time = 373
  by: Overall equipment effectiveness (OEE)  (A = (T - D)/T, P = ct n / (T - D), Q = good/n, OEE =
A P Q (= good x ct / T))
  planned_time = 420
  downtime = 47
  ideal_cycle_time = 1
  total_count = 300
  good_count = 285
  assumes: downtime < planned time; good <= total; performance <= 1
```

What the worked example means: This calculation determines the Overall Equipment Effectiveness by multiplying three key factors: availability, performance, and quality. Availability measures how much time the machine was actually running compared to the planned production time, calculated as run time divided by planned time. Performance assesses how fast the machine ran relative to its ideal speed, using the ratio of

ideal cycle time times total count over run time. Quality evaluates the proportion of good units produced out of the total units. The final result, 0.678571, represents the combined efficiency of these three dimensions, indicating that for every unit of planned time, only a fraction resulted in good parts at ideal speed.

Standard time from a time study, `ie_standard_time`

Normal time (mean observed time x performance rating) and standard time with a personal, fatigue and delay allowance.

$$NT = \bar{t} \times PR, \quad ST = NT(1 + A) \text{ or } \frac{NT}{1 - A}$$

symbol		meaning	SI unit
`observed_times`	in	observed cycle times	,
`rating`	in	performance rating (decimal; 110% = 1.1)	,
`allowance`	in	allowance fraction (decimal; 15% = 0.15)	,
`basis`	in	job: ST = NT (1 + A); shift: ST = NT / (1 - A), A a share of the workday	,
`standard_time`	out	standard time	,
`normal_time`	out	mean x rating	,
`mean_observed_time`	out	mean observed time	,

Applies when: observed times > 0; rating > 0; 0 <= allowance < 1; the allowance basis must be stated.

```
$ industrial-engineering solve ie_standard_time "observed_times=[4.2, 4.5, 4, 4.3, 4.5]" rating=1.1
allowance=0.15 basis=job
standard_time = 5.4395
normal_time = 4.73
mean_observed_time = 4.3
  by: Standard time from a time study  (NT = mean x rating; ST = NT (1 + A) on a job basis or NT /
(1 - A) on a workday basis)
  rating = 1.1
  allowance = 0.15
  basis = job
  assumes: observed times > 0; rating > 0; 0 <= allowance < 1; the allowance basis must be stated
```

What the worked example means: This calculation determines the standard time for a task by first finding the mean observed time, which is 4.3. This average is multiplied by the performance rating of 1.1 to derive the normal time of 4.73. Finally, an allowance of 0.15 is applied using the job basis to produce the standard time of 5.4395, representing the total time required to complete the job under normal working conditions.

Work sampling sample size, `ie_work_sampling_sample_size`

Random observations needed to estimate the fraction of time in an activity within +/- h (absolute) at a confidence level: n = z^2 p (1 - p) / h^2.

$$n = \frac{z^2\, p(1 - p)}{h^2}$$

symbol		meaning	SI unit
`proportion`	in	estimated fraction of time in the activity (decimal)	,
`accuracy`	in	absolute accuracy (decimal; 0.05 = +/- 5 points)	,
`confidence`	in	confidence level (decimal)	,
`sample_size`	out	observations needed (rounded up)	,
`exact`	out	z^2 p (1 - p) / h^2	,
`z`	out	two-sided z	,

Applies when: 0 < p < 1; 0 < h < 1; 0 < confidence < 1.

```
$ industrial-engineering solve ie_work_sampling_sample_size proportion=0.2 accuracy=0.05
confidence=0.95
sample_size = 246
exact = 245.8533645
z = 1.959963985
  by: Work sampling sample size  (n = z^2 p (1 - p) / h^2)
  proportion = 0.2
  accuracy = 0.05
  confidence = 0.95
  assumes: 0 < p < 1; 0 < h < 1; 0 < confidence < 1
```

What the worked example means: This calculation determines the number of random observations needed to estimate the fraction of time spent on an activity. It uses the formula n equals z squared times p times one minus p divided by h squared, where p is the estimated proportion, h is the desired absolute accuracy, and z corresponds to the confidence level. The resulting exact value is 245.853, which rounds up to 246 observations. This ensures that the true proportion lies within plus or minus 0.05 of the estimate with 0.95 confidence.

Questions and reports: support@ai999x.com. AI999X, The Future Is Self-Improving.